



Sanchit Jain

Legend

I am a computer science faculty, having a teaching experience of more than 8 years. I have a very sound experience in teaching students.

43M Watch mins

3M Watch mins (last 30 days)

20K Followers

2K Dedications



Educator highlights

-  **Works at Knowledge Gate**
-  **Studied at Delhi Technological University**
-  **Have qualified gate Have experience of more than 8 years Have a YouTube channel with 5 lakh follower**
-  **Lives in Ghaziabad, Uttar Pradesh, India**
-  **Unacademy Educator since 19th July, 2019**
-  **844,388 live minutes taught in last 30 days**
-  **Knows Punjabi, Hinglish, Hindi and English**

Months Wise Calendar for GATE-2022 (Unacademy Plus) (Sanchit Jain)

Month	Morning Class (7:00am – 9:00am)	Evening Class (7:00pm – 9:00pm)
Feb-2021	DS & Programming (15-Feb to 23-Mar) (56 Hours) Algo (24-Mar to 26-April) (38 Hours) CN(29-April to 15-June) (54 Hours) TOC (26-May to 05-July) (58 Hours) Compiler (7-July to 5-Aug) (34 Hours)	DM (22-Feb to 27-Mar) (50 Hours) DBMS (29-Mar to 30-April) (50 Hours) DE (7-May to 1-June) (40 Hours) COA (23-June to 20-July) (38 Hours) OS (21-July to 24-Aug) (58 Hours)
Mar-2021		
Apr-2021		
May-2021		
Jun-2021		
Jul-2021		
Aug-2021	DM (11-Aug to 20-Sept) (50 Hours) DBMS (22-Sept to 29-Oct) (50 Hours) DE (3-Nov to 23-Nov) (40 Hours) COA (24-Nov to 13-Dec) (38 Hours) OS (15-Dec to 11-Jan) (58 Hours) TOC (12-Jan to 12-Feb) (58 Hours)	DS & Programming (25-Aug to 5-Oct) (56 Hours) Algo (6-Oct to 1-Nov) (38 Hours) CN(10-Nov to 5 Jan) (54 Hours) Compiler (26-Jan to 15-Feb) (34 Hours)
Sep-2021		
Oct-2021		
Nov-2021		
Dec-2021		
Jan-2022		
Feb-2022		

Date	Day	Subject-1	Topic	Time	Duration
03-11-2021	Wednesday	Digital Electronics	Boolean Algebra & Expression Part-1	7:00 am to 9:00 am	2
04-11-2021	Thursday	Digital Electronics	Boolean Algebra & Expression Part-2	7:00 am to 9:00 am	2
05-11-2021	Friday				
06-11-2021	Saturday				
07-11-2021	Sunday				
08-11-2021	Monday	Digital Electronics	Simplification of Boolean Algebra Part-1	7:00 am to 9:00 am	2
09-11-2021	Tuesday	Digital Electronics	Doubt Clearing Session	7:00 am to 9:00 am	2
10-11-2021	Wednesday	Digital Electronics	Simplification of Boolean Algebra Part-2	7:00 am to 9:00 am	2
11-11-2021	Thursday	Digital Electronics	Simplification of Boolean Algebra Part-3	7:00 am to 9:00 am	2
12-11-2021	Friday	Digital Electronics	Combinational Circuits Part-1	7:00 am to 9:00 am	2
13-11-2021	Saturday	Digital Electronics	Doubt Clearing Session	7:00 am to 9:00 am	2
14-11-2021	Sunday	Digital Electronics	Combinational Circuits Part-2	7:00 am to 9:00 am	2
15-11-2021	Monday	Digital Electronics	Combinational Circuits Part-3	7:00 am to 9:00 am	2
16-11-2021	Tuesday	Digital Electronics	Sequential Circuits Part-1	7:00 am to 9:00 am	2
17-11-2021	Wednesday	Digital Electronics	Doubt Clearing Session	7:00 am to 9:00 am	2
18-11-2021	Thursday	Digital Electronics	Sequential Circuits Part-2	7:00 am to 9:00 am	2
19-11-2021	Friday	Digital Electronics	Sequential Circuits Part-3	7:00 am to 9:00 am	2
20-11-2021	Saturday	Digital Electronics	Number System Part-1	7:00 am to 9:00 am	2
21-11-2021	Sunday	Digital Electronics	Doubt Clearing Session	7:00 am to 9:00 am	2
22-11-2021	Monday	Digital Electronics	Number System Part-2	7:00 am to 9:00 am	2
23-11-2021	Tuesday	Digital Electronics	Floating Point Representation	7:00 am to 9:00 am	2

Date	Day	Subject-2	Topic	Time	Duration
10-11-2021	Wednesday	CN	Basics of Computer Networks Part-1	6:00 pm to 8:00 pm	2
11-11-2021	Thursday	CN	Basics of Computer Networks Part-2	6:00 pm to 8:00 pm	2
15-11-2021	Monday	CN	Basics of Computer Networks Part-3	6:00 pm to 8:00 pm	2
16-11-2021	Tuesday	CN	Doubt Clearing Session	6:00 pm to 8:00 pm	2
17-11-2021	Wednesday	CN	Access Control Part-1	6:00 pm to 8:00 pm	2
18-11-2021	Thursday	CN	Access Control Part-2	6:00 pm to 8:00 pm	2
22-11-2021	Monday	CN	Flow Control Part-1	6:00 pm to 8:00 pm	2
23-11-2021	Tuesday	CN	Doubt Clearing Session	6:00 pm to 8:00 pm	2
24-11-2021	Wednesday	CN	Error Control Part-1	6:00 pm to 8:00 pm	2
25-11-2021	Thursday	CN	Error Control Part-2	6:00 pm to 8:00 pm	2
29-11-2021	Monday	CN	Framing and Ethernet	6:00 pm to 8:00 pm	2
30-11-2021	Tuesday	CN	Doubt Clearing Session	6:00 pm to 8:00 pm	2
01-12-2021	Wednesday	CN	Network Layer protocol Part-1	6:00 pm to 8:00 pm	2
02-12-2021	Thursday	CN	Network Layer protocol Part-2	6:00 pm to 8:00 pm	2
06-12-2021	Monday	CN	Network Layer protocol Part-3	6:00 pm to 8:00 pm	2
07-12-2021	Tuesday	CN	Doubt Clearing Session	6:00 pm to 8:00 pm	2
13-12-2021	Monday	CN	IP addressing and routing and Routing Part-1	6:00 pm to 8:00 pm	2
14-12-2021	Tuesday	CN	IP addressing and routing and Routing Part-2	6:00 pm to 8:00 pm	2
20-12-2021	Monday	CN	Transport Layer Part-1	6:00 pm to 8:00 pm	2
21-12-2021	Tuesday	CN	Doubt Clearing Session	6:00 pm to 8:00 pm	2
27-12-2021	Monday	CN	Transport Layer Part-2	6:00 pm to 8:00 pm	2
28-12-2021	Tuesday	CN	Transport Layer Part-3	6:00 pm to 8:00 pm	2
03-01-2022	Monday	CN	Application Layer	6:00 pm to 8:00 pm	2
04-01-2022	Tuesday	CN	Doubt Clearing Session	6:00 pm to 8:00 pm	2
05-01-2022	Wednesday	CN	crypto	6:00 pm to 8:00 pm	2

GATE & ESE

Plus

All courses

All languages

All topics

7:00 AM

HINDI CS & IT

Complete Course on Digital Electronics for GATE

Lesson 2 - Today, 7:00 AM

Sanchit Jain

HINGLISH PRACTICE & STRATEGY

FAQ Handbook for GATE (CS & IT)

Lesson 7 - Nov 10, 2021 8:31 PM

Ravindrababu Ravula

6:05 PM

HINDI CS & IT

Complete Course on Algorithms for GATE

Lesson 18 - Today, 6:05 PM

Sanchit Jain

HINGLISH CS & IT

Complete Course on Computer Networks for GATE

Starts on Nov 10, 2021 - 28 lessons

Sanchit Jain

HINGLISH CS & IT

Complete Course on DBMS for GATE

Ended on Oct 26, 2021 - 25 lessons

Sanchit Jain

HINDI CS & IT

Complete Course on Programming & Data Structures for GATE

Ended on Oct 12, 2021 - 28 lessons

Sanchit Jain

HINDI CS & IT

Complete Course on Discrete Mathematics for GATE

Ended on Oct 30, 2021 - 27 lessons

Sanchit Jain

HINDI CS & IT

Foundation Course on Operating Systems for GATE 2022

Ended on Aug 23, 2021 - 30 lessons

Sanchit Jain

HINDI CS & IT

Complete Course on Compiler Design for GATE

Ended on Aug 13, 2021 - 18 lessons

Sanchit Jain

HINDI CS & IT

Complete Course on Computer Architecture for GATE 2022

Ended on Jul 20, 2021 - 19 lessons

Sanchit Jain

HINGLISH CS & IT

Foundation Course on Theory of Computation for GATE 2022

Ended on Jul 5, 2021 - 29 lessons

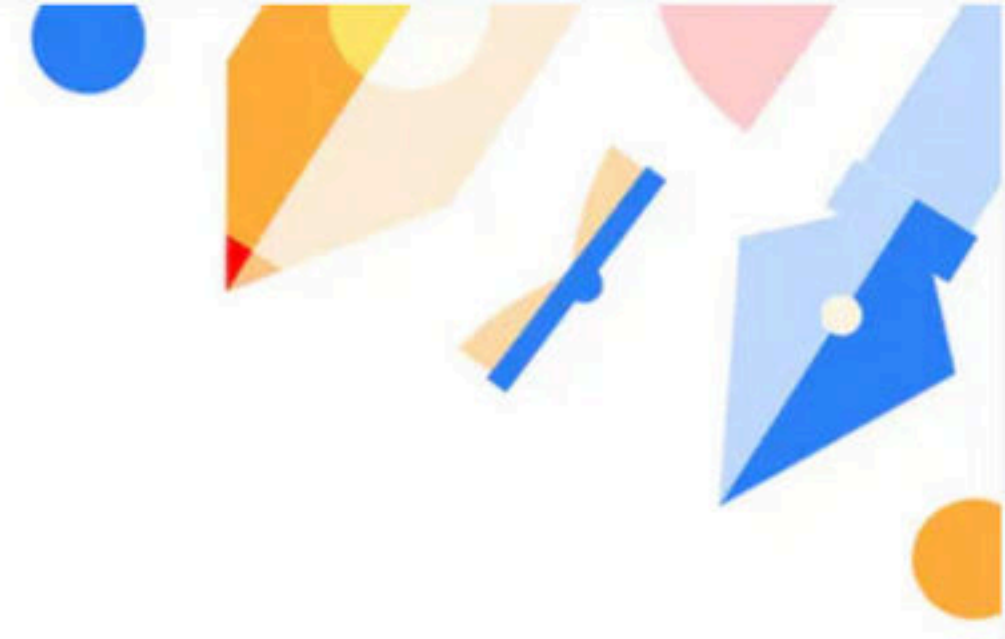
Sanchit Jain

HINDI CS & IT

Foundation Course on Digital Electronics

Ended on Jun 1, 2021 - 20 lessons

Sanchit Jain

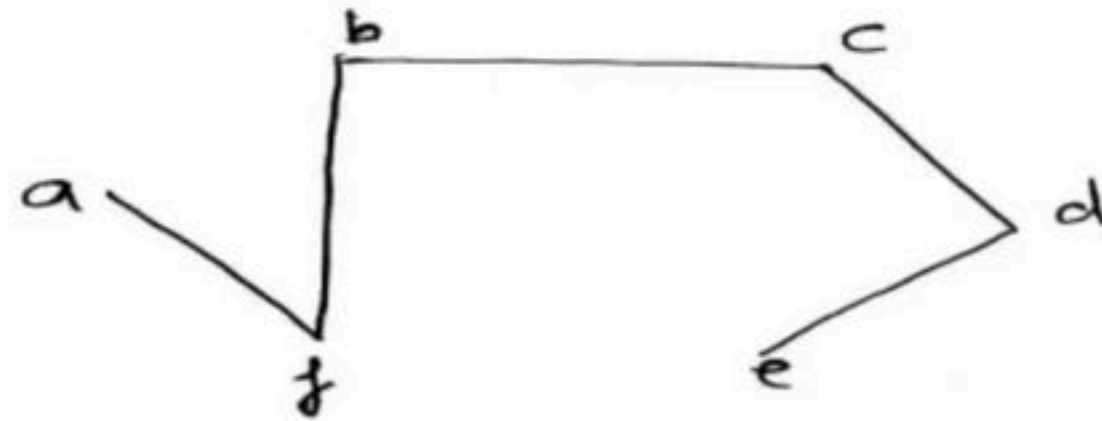
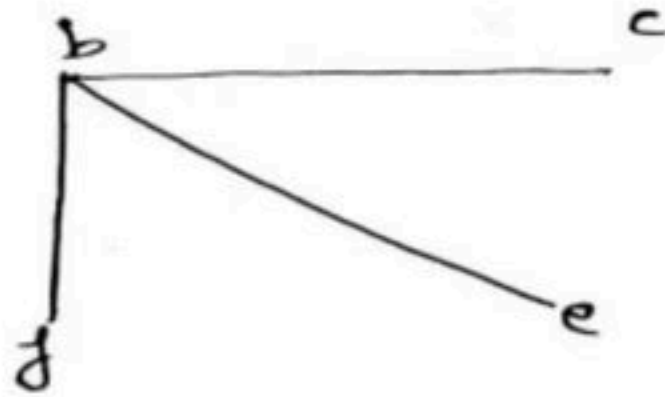
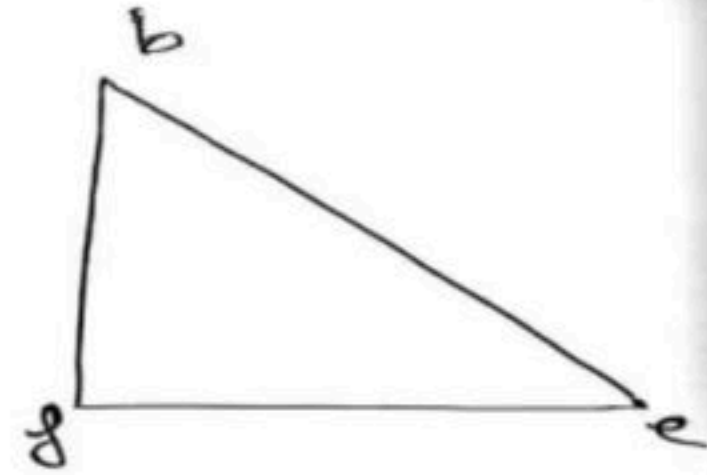
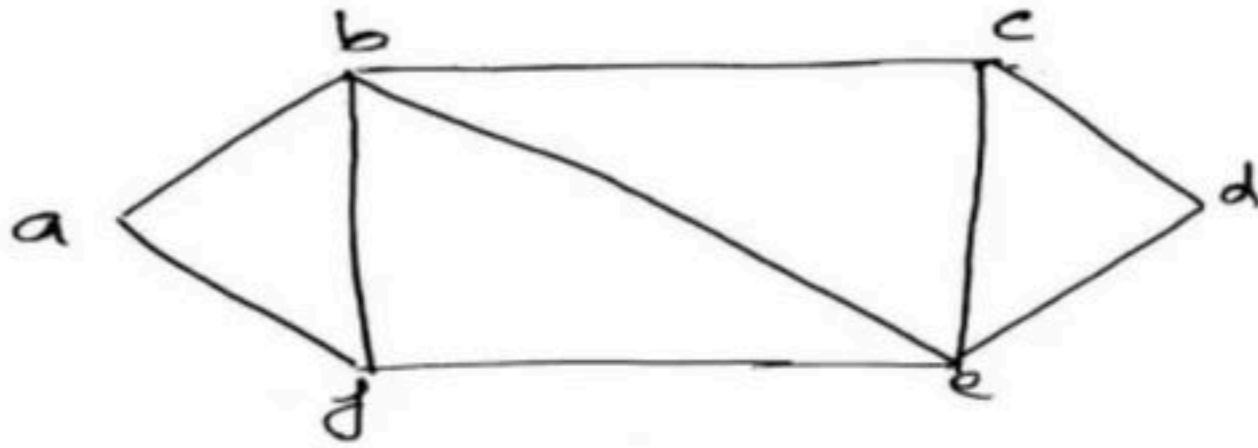


Shortest Path - II & Doubt Clearing Session

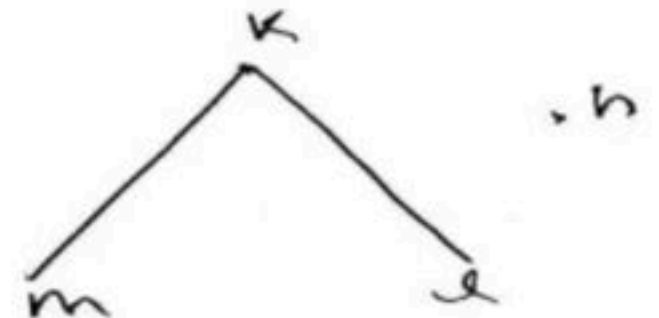
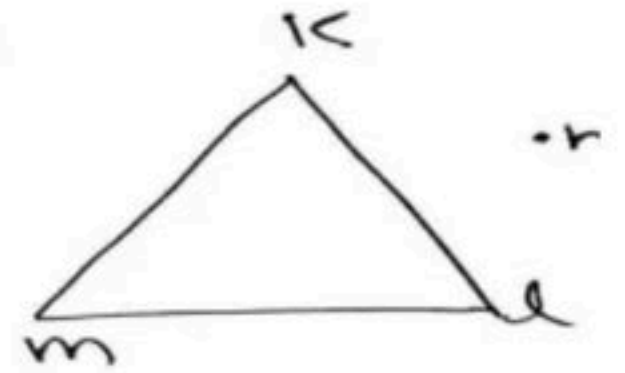
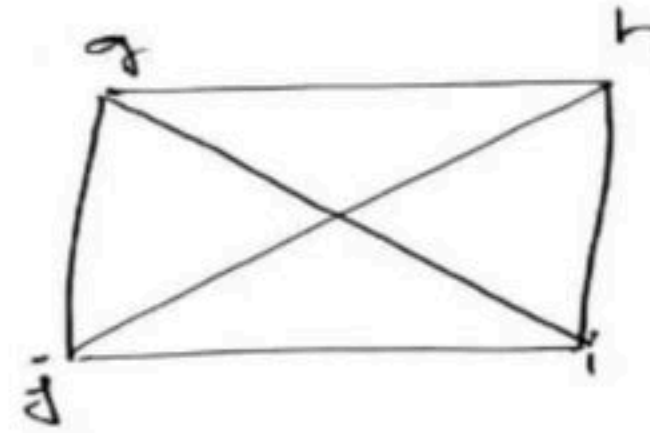
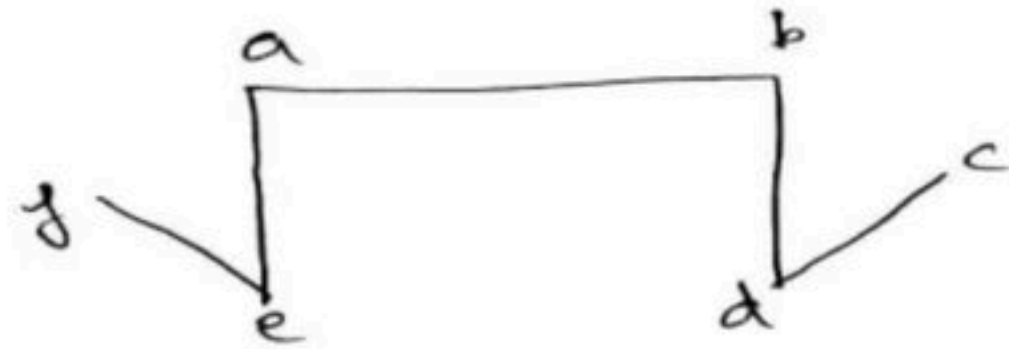
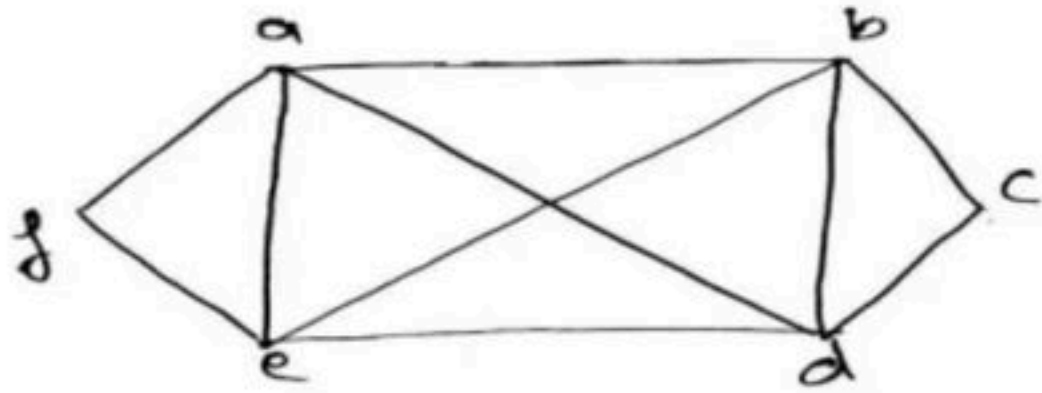
Complete Course on Algorithms for GATE

Spanning tree

- A tree T is said to be spanning tree of a connected graph G , if T is a subgraph of G and T contains all vertices of G .



- In general, a graph may have several spanning trees, but a graph that is not connected will not contain a spanning tree (but see Spanning forests below).

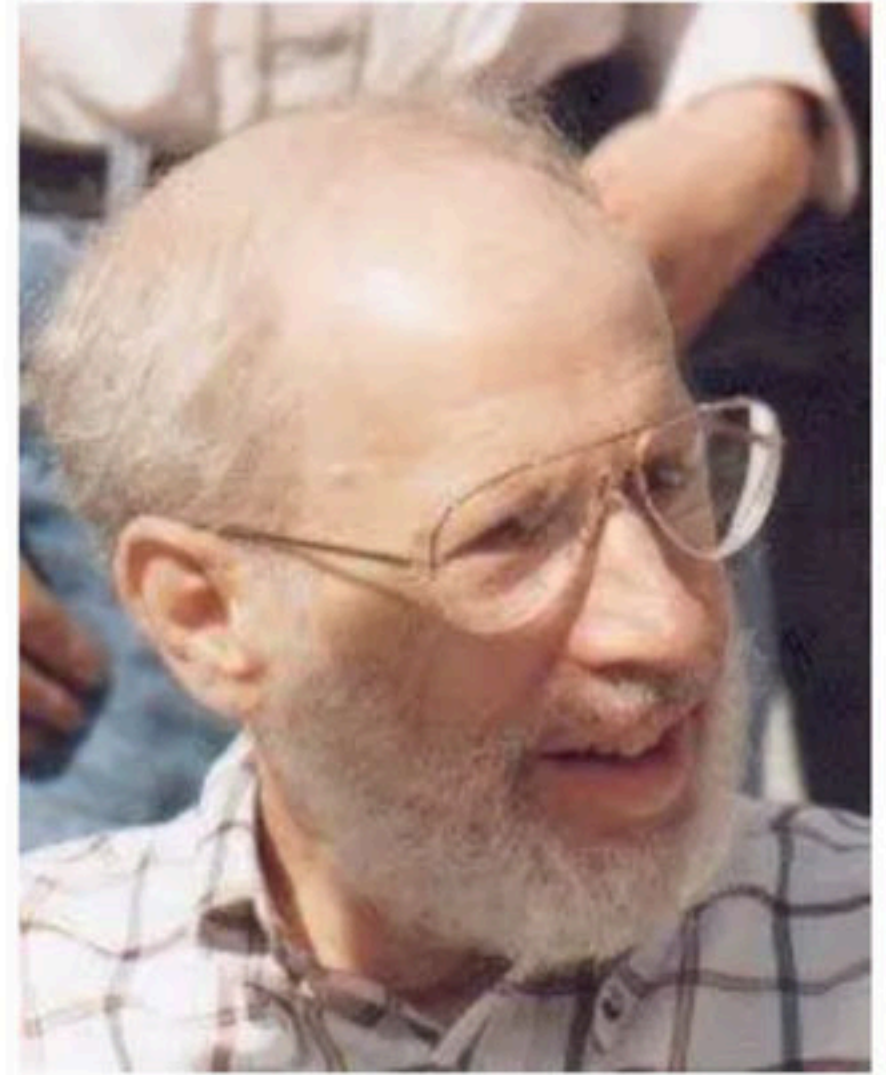


- A **minimum spanning tree (MST)** or **minimum weight spanning tree** is a subset of the edges of a connected, edge-weighted undirected graph that connects all the vertices together, without any cycles and with the minimum possible total edge weight.
- That is, it is a spanning tree whose sum of edge weights is as small as possible. More generally, any edge-weighted undirected graph (not necessarily connected) has a **minimum spanning forest**, which is a union of the minimum spanning trees for its connected components.

Break

Kruskal Algorithm

- **Joseph Bernard Kruskal, Jr.** was an American mathematician, statistician, computer scientist and psychometrician.



Kruskal

Minimum_Spanning_Tree (G, w)

```
{  
    A  $\leftarrow \phi$   
    For each vertex  $v \in V(G)$   
    {  
        do Make_Set(v)  
    }  
    Sort the edges of E into non-decreasing order by weight w  
    for each edge  $(u, v) \in E$ , then in non-decreasing order by weights  
    {  
        if (Find_Set(u)  $\neq$  Find_Set(v))  
        {  
            A  $\leftarrow A \cup \{(u, v)\}$   
            UNION (u, v)  
        }  
    }  
    Return A  
}
```

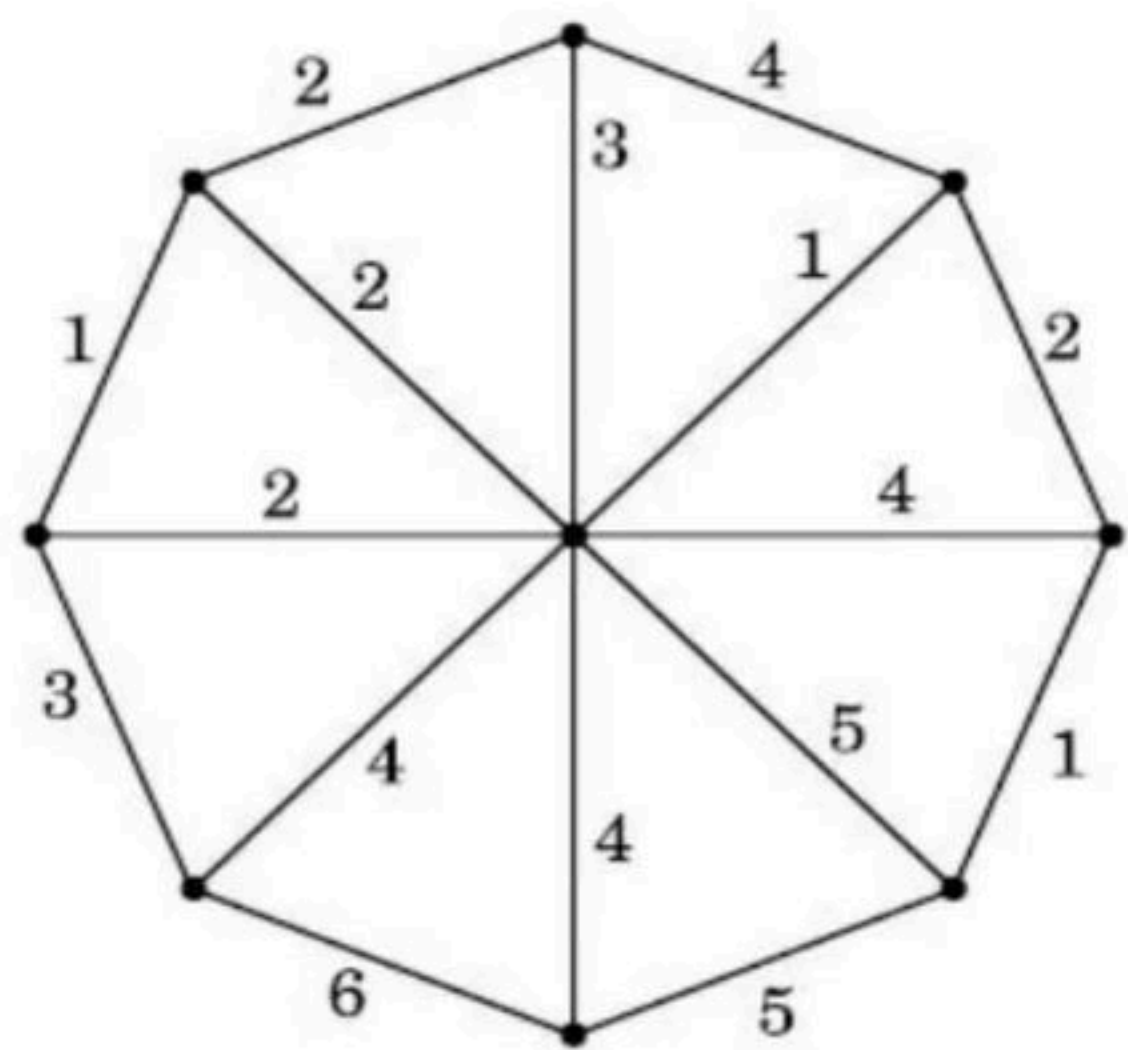
MST-KRUSKAL(G, w)

```
1 A = ∅ —  $\emptyset$  (1)  
2 for each vertex  $v \in G.V$   
3   MAKE-SET( $v$ )  
4 sort the edges of  $G.E$  into nondecreasing order by weight  $w$   
5 for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  
6   if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )  
7      $A = A \cup \{(u, v)\}$   
8     UNION( $u, v$ )  
9 return  $A$ 
```

$\pm \lg E$

The running time of Kruskal's algorithm for a graph $G = (V, E)$ depends on how we implement the disjoint-set data structure. We assume that we use the disjoint-set-forest implementation of Section 21.3 with the union-by-rank and path-compression heuristics, since it is the asymptotically fastest implementation known. Initializing the set A in line 1 takes $O(1)$ time, and the time to sort the edges in line 4 is $O(E \lg E)$. (We will account for the cost of the $|V|$ MAKE-SET operations in the for loop of lines 2–3 in a moment.) The for loop of lines 5–8 performs $O(E)$ FIND-SET and UNION operations on the disjoint-set forest. Along with the $|V|$ MAKE-SET operations, these take a total of $O((V + E)\alpha(V))$ time, where α is the very slowly growing function defined in Section 21.4. Because we assume that G is connected, we have $|E| \geq |V| - 1$, and so the disjoint-set operations take $O(E\alpha(V))$ time. Moreover, since $\alpha(|V|) = O(\lg V) = O(\lg E)$, the total running time of Kruskal's algorithm is $O(E \lg E)$. Observing that $|E| < |V|^2$, we have $\lg |E| = O(\lg V)$, and so we can restate the running time of Kruskal's algorithm as $O(E \lg V)$.

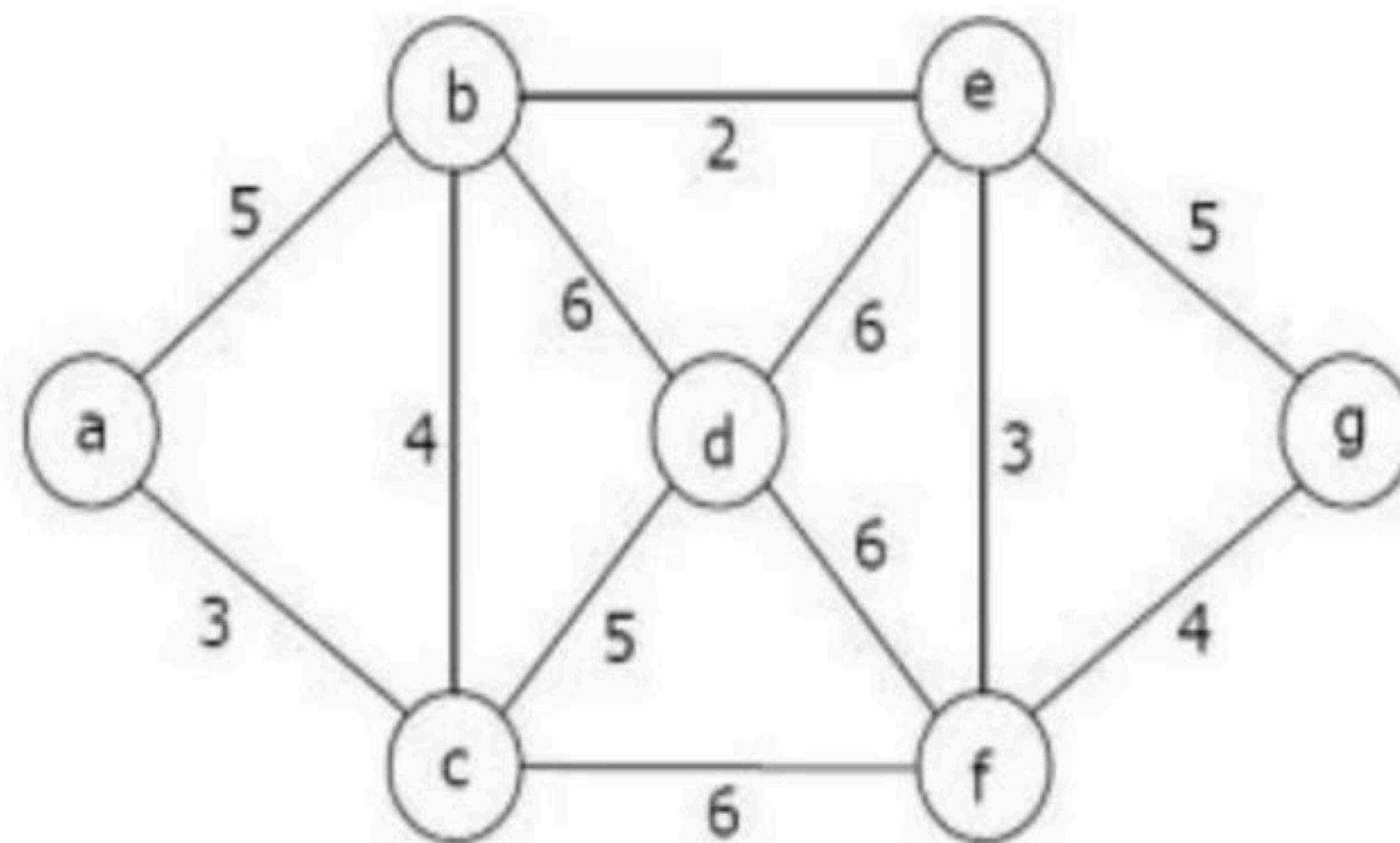
Q Consider the graph shown below (NET-DEC-2018)



Use Kruskal's algorithm to find the minimum spanning tree of the graph. The weight of this minimum spanning tree is

- a) 17
- b) 14
- c) 16
- d) 13

Q Consider the following graph:



Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm? (Gate-2009)(2 Marks)

(A) (b, e), (e, f), (a, c), (b, c), (f, g), (c, d)

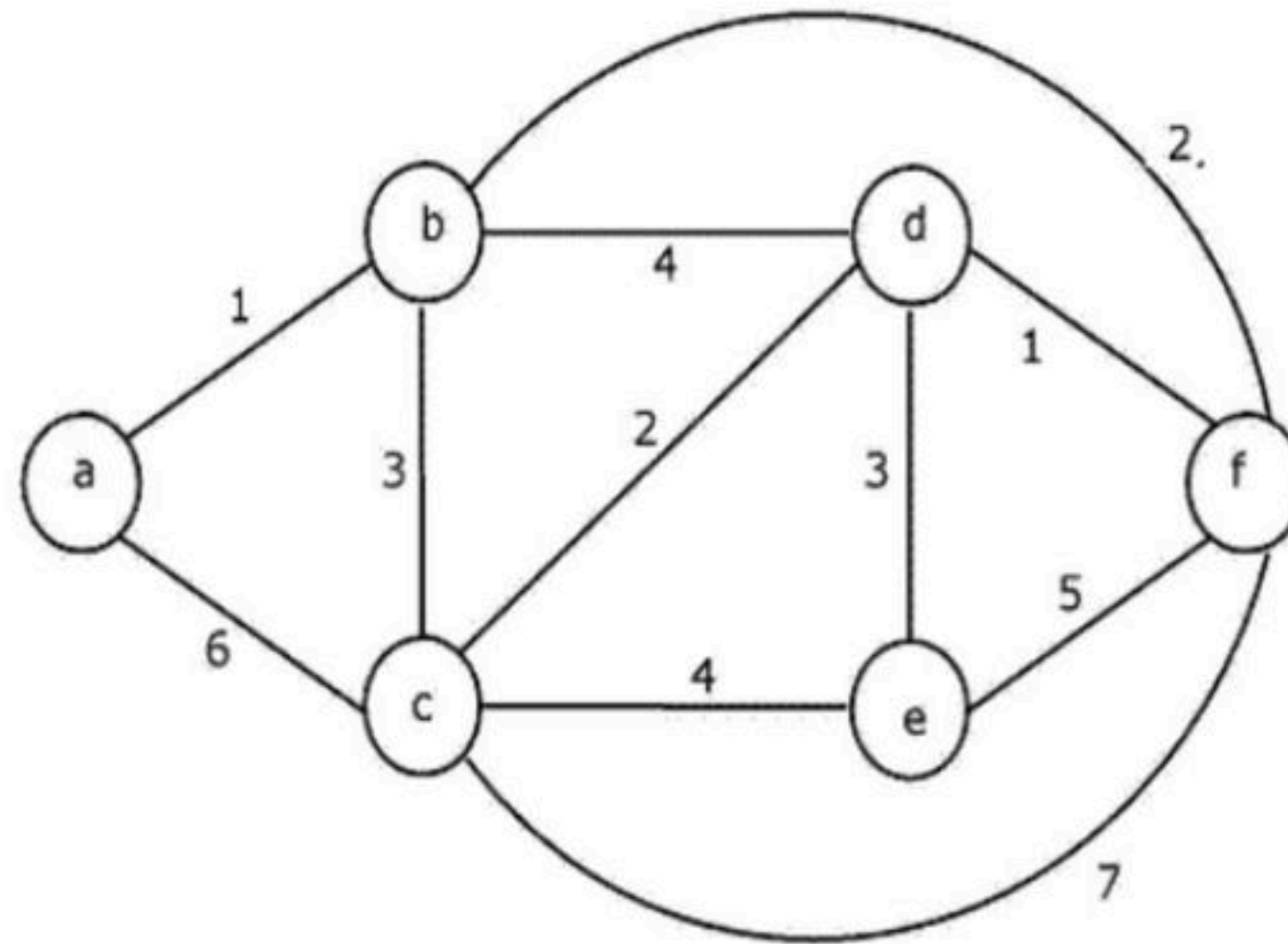
(B) (b, e), (e, f), (a, c), (f, g), (b, c), (c, d)

(C) (b, e), (a, c), (e, f), (b, c), (f, g), (c, d)

(D) (b, e), (e, f), (b, c), (a, c), (f, g), (c, d)

Q Consider the following graph:

Which one of the following cannot be the sequence of edges added, in that order, to a minimum spanning tree using Kruskal's algorithm? **(Gate-2006) (2 Marks)**



(A) (a—b), (d—f), (b—f), (d—c), (d—e)

(C) (d—f), (a—b), (d—c), (b—f), (d—e)

(B) (a—b), (d—f), (d—c), (b—f), (d—e)

(D) (d—f), (a—b), (b—f), (d—e), (d—c)

Break

- **Robert Clay Prim** (born September 25, 1921 in Sweetwater, Texas) is an American mathematician and computer scientist.



Minimum_Spanning_Tree (G, W, R)

{

 for each $u \in V(G)$

 {

$\text{key}[u] \leftarrow \infty$

$\Pi[u] \leftarrow \text{NIL}$

 }

$\text{Key}[r] \leftarrow 0$

$Q \leftarrow V[G]$

 While ($Q \neq \phi$)

 {

$u \leftarrow \text{Extract-Min}(Q)$

 For each $v \in \text{adj}[u]$

 {

 if ($v \in Q$ and $w(u, v) < \text{key}[v]$)

 {

$\Pi[v] \leftarrow u$

$\text{key}[v] \leftarrow w(u, v)$

 }

 }

 }

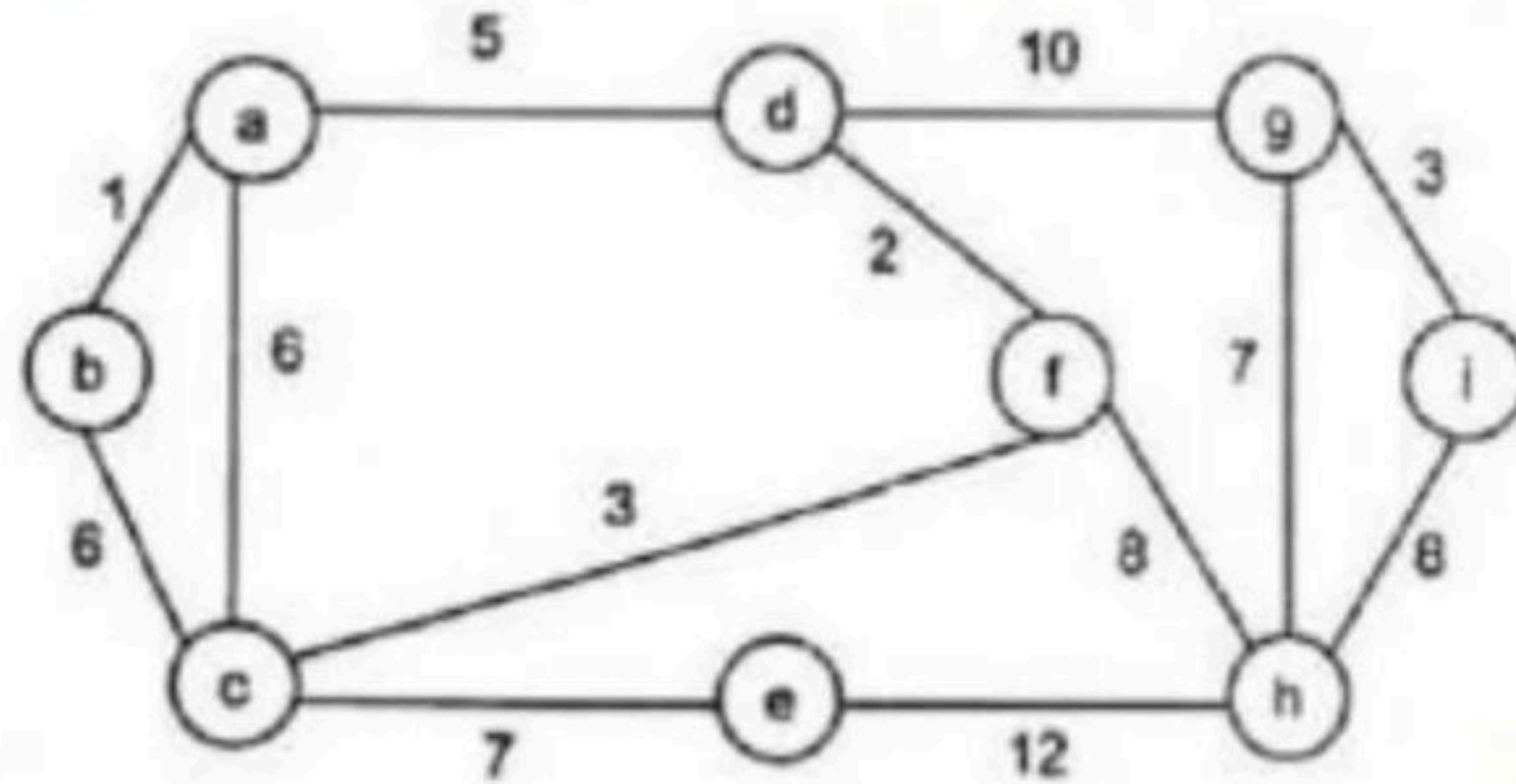
}

MST-PRIM(G, w, r)

```
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
```

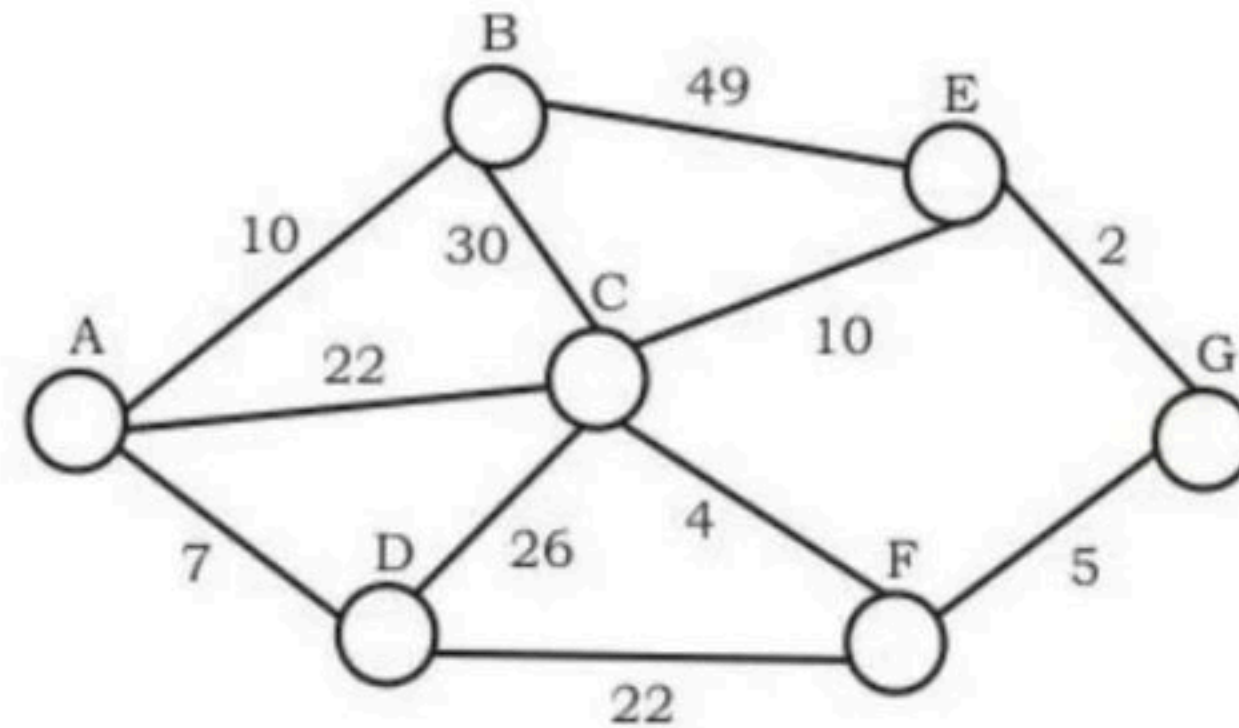
The running time of Prim's algorithm depends on how we implement the min-priority queue Q . If we implement Q as a binary min-heap (see Chapter 6), we can use the BUILD-MIN-HEAP procedure to perform lines 1–5 in $O(V)$ time. The body of the while loop executes $|V|$ times, and since each EXTRACT-MIN operation takes $O(\lg V)$ time, the total time for all calls to EXTRACT-MIN is $O(V \lg V)$. The for loop in lines 8–11 executes $O(E)$ times altogether, since the sum of the lengths of all adjacency lists is $2|E|$. Within the for loop, we can implement the test for membership in Q in line 9 in constant time by keeping a bit for each vertex that tells whether or not it is in Q , and updating the bit when the vertex is removed from Q . The assignment in line 11 involves an implicit DECREASE-KEY operation on the min-heap, which a binary min-heap supports in $O(\lg V)$ time. Thus, the total time for Prim's algorithm is $O(V \lg V + E \lg V) = O(E \lg V)$, which is asymptotically the same as for our implementation of Kruskal's algorithm.

Q For the undirected, weighted graph given below, which of the following sequences of edges represents a correct execution of Prim's algorithm to construct a Minimum Spanning Tree?
(Gate-2008) (2 Marks)



- (A)** (a, b), (d, f), (f, c), (g, i), (d, a), (g, h), (c, e), (f, h)
- (B)** (c, e), (c, f), (f, d), (d, a), (a, b), (g, h), (h, f), (g, i)
- (C)** (d, f), (f, c), (d, a), (a, b), (c, e), (f, h), (g, h), (g, i)
- (D)** (h, g), (g, i), (h, f), (f, c), (f, d), (d, a), (a, b), (c, e)

Q Consider the undirected graph below:

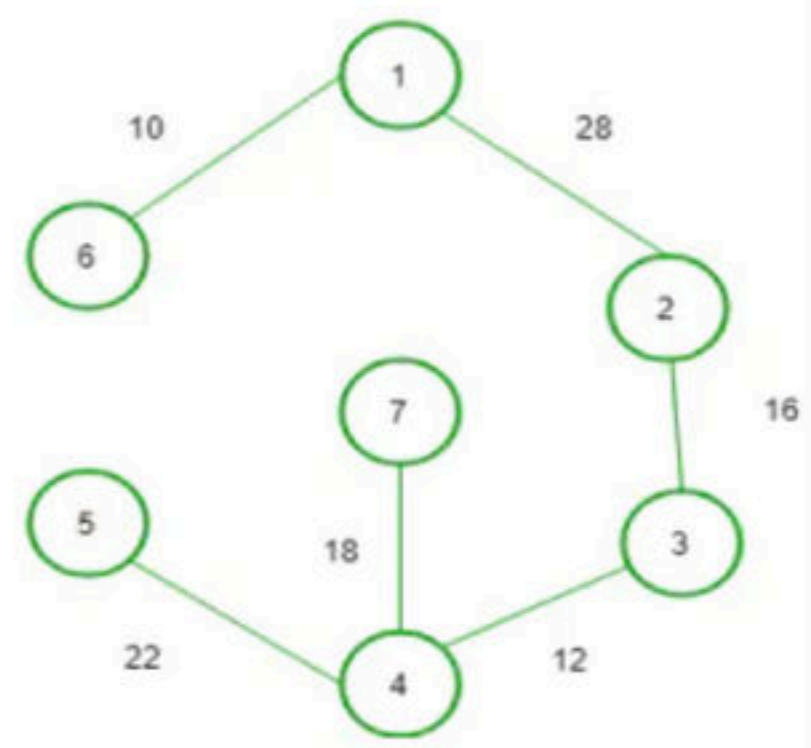
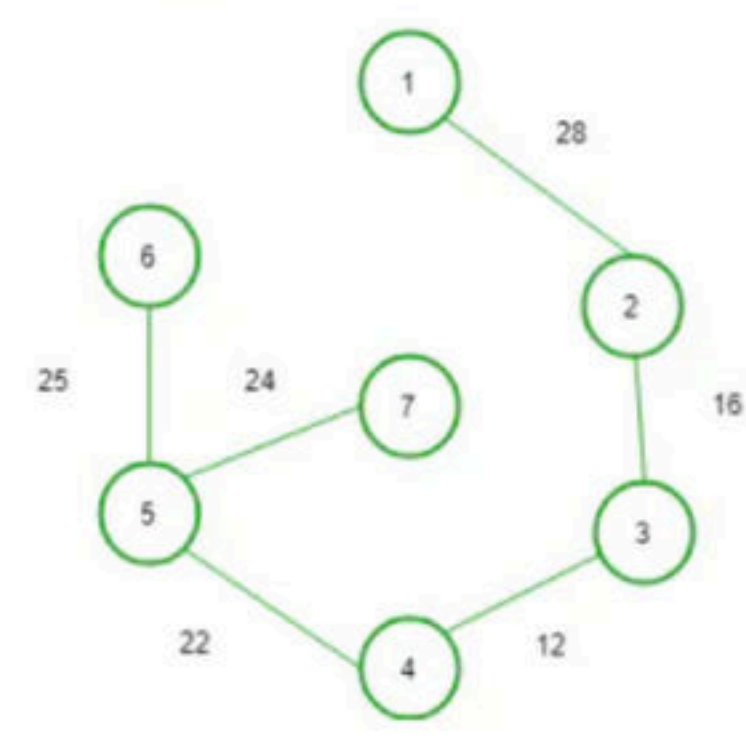
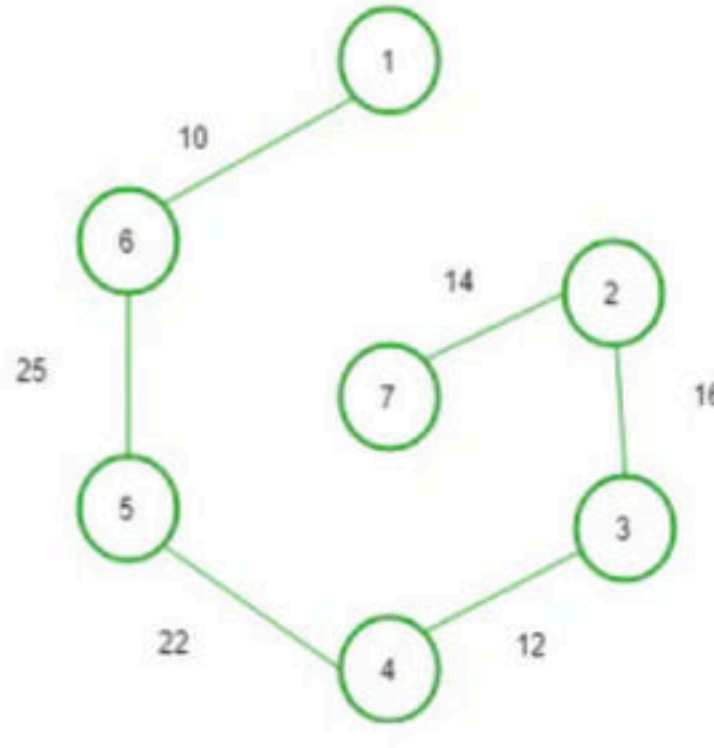
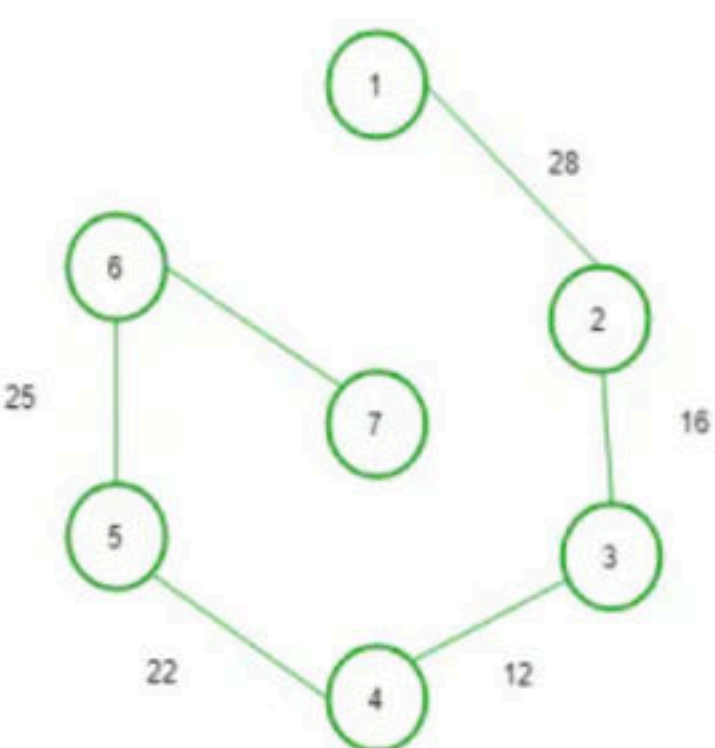
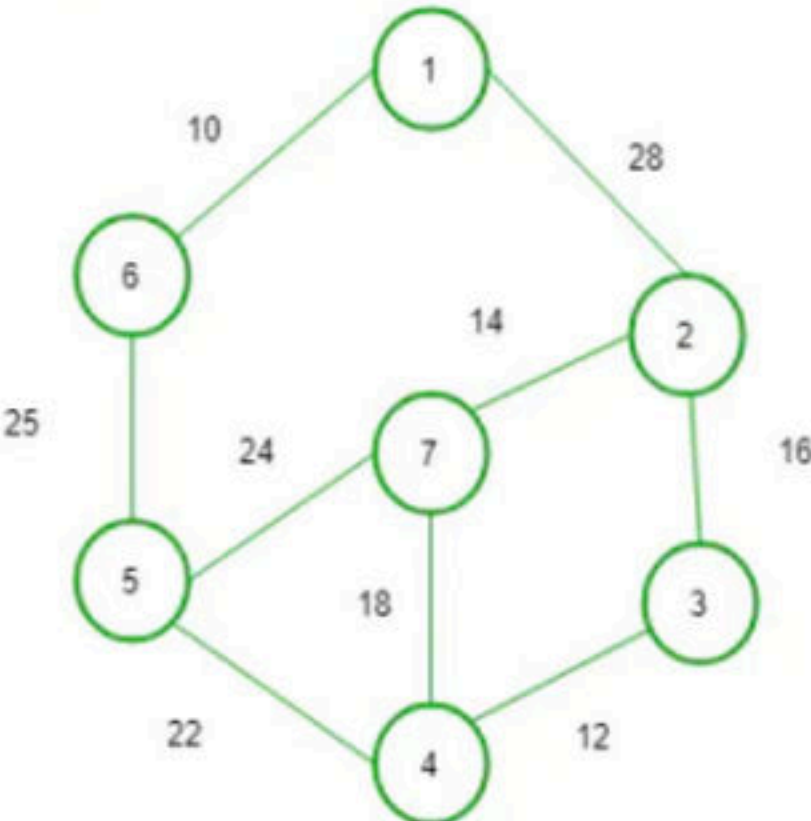


Using Prim's algorithm to construct a minimum spanning tree starting with node A, which one of the following sequences of edges represents a possible order in which the edges would be added to construct the minimum spanning tree? **(Gate-2004) (2 Marks)**

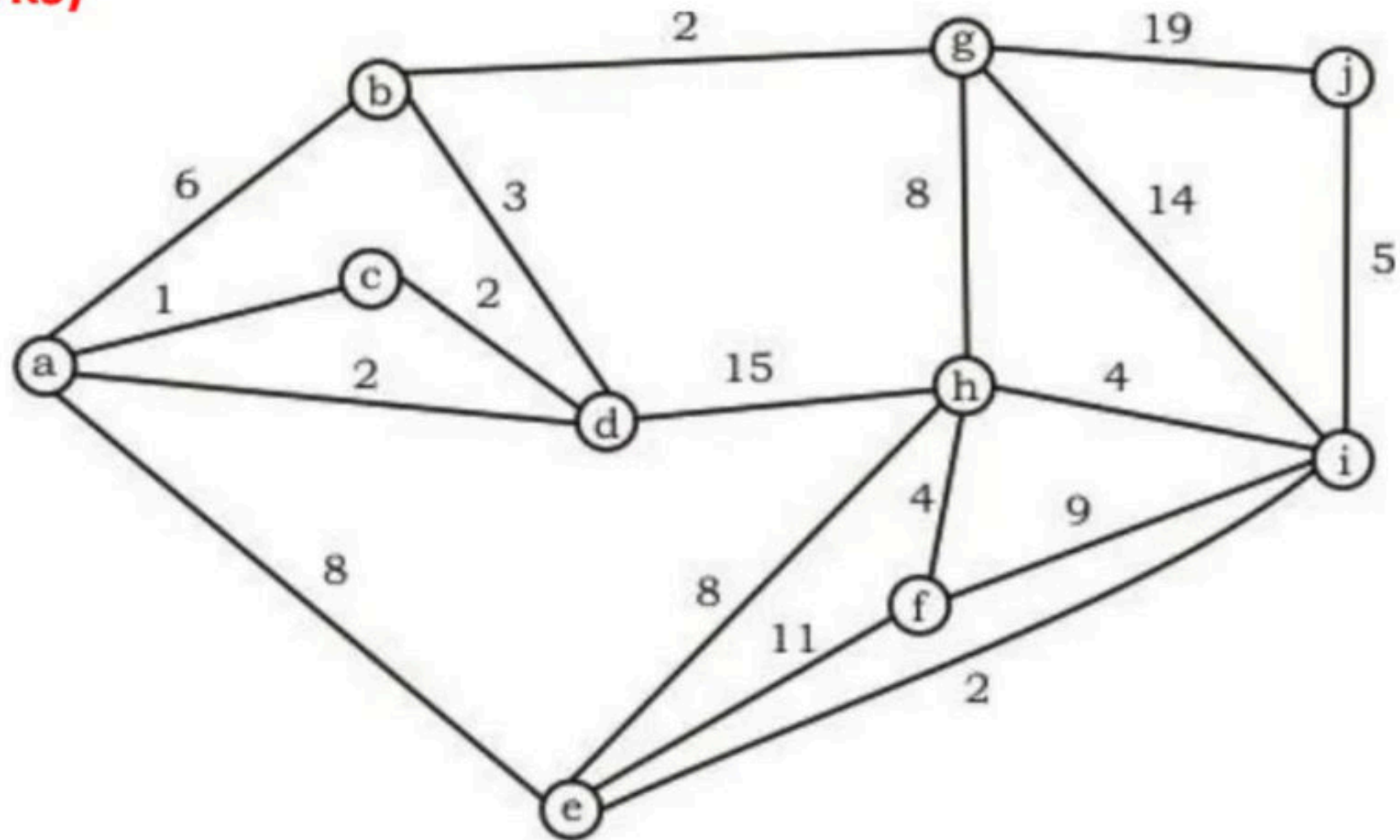
- (A)** (E, G), (C, F), (F, G), (A, D), (A, B), (A, C)
- (B)** (A, D), (A, B), (A, C), (C, F), (G, E), (F, G)
- (C)** (A, B), (A, D), (D, F), (F, G), (G, E), (F, C)
- (D)** (A, D), (A, B), (D, F), (F, C), (F, G), (G, E)

Break

Q Consider the following graph (NET-JUNE-2015)
Its minimum cost spanning tree is -



Q What is the weight of a minimum spanning tree of the following graph? **(Gate-2003) (2 Marks)**



(A) 29

(B) 31

(C) 38

(D) 41

Break

Q Let G be any connected, weighted, undirected graph:

I. G has a unique minimum spanning tree if no two edges of G have the same weight.

II. G has a unique minimum spanning tree if, for every cut G , there is a unique minimum weight edge crossing the cut.

Which of the above two statements is/are TRUE? (**Gate-2019**) (**2 Marks**)

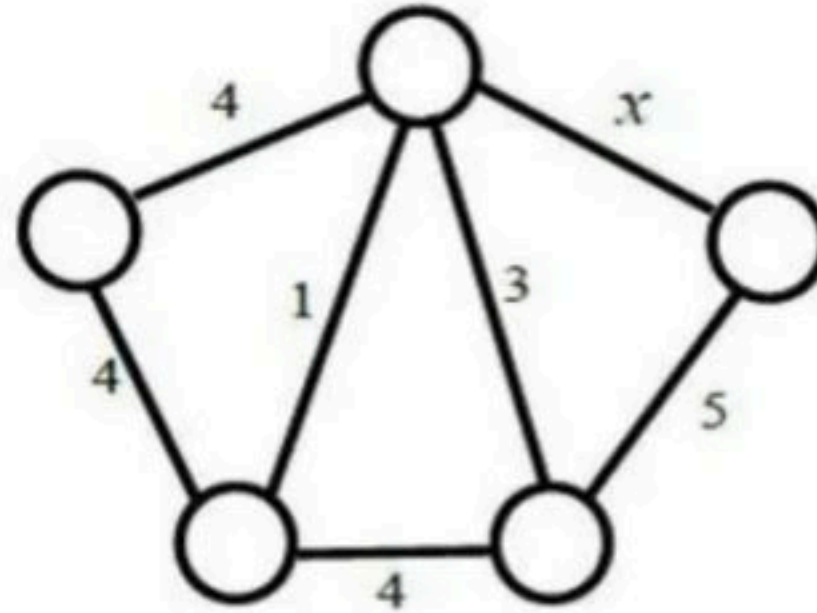
a) Neither I nor II

b) I only

c) II only

d) Both I and II

Q Consider the following undirected graph

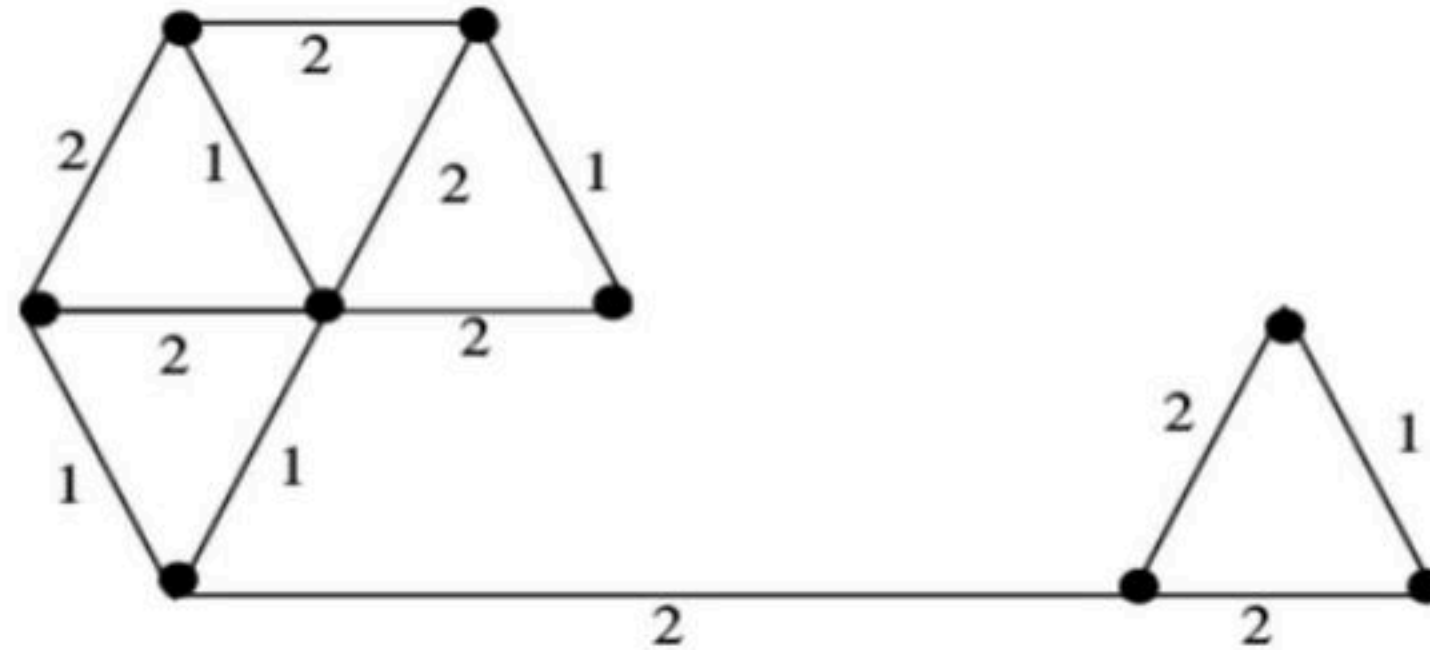


Choose a value for x that will maximize the number of minimum weights spanning trees (MWSTs) of G . The number of MWSTs of G for this value of x is _____. (**Gate-2018**) (**2 Marks**)

Q Let G be an undirected connected graph with distinct edge weight. Let $e_{\max}$ be the edge with maximum weight and $e_{\min}$ the edge with minimum weight. Which of the following statements is false? **(Gate-2000) (2 Marks) (NET-NOV-2017)**

- (A)** Every minimum spanning tree of G must contain $e_{\min}$
- (B)** If $e_{\max}$ is in a minimum spanning tree, then its removal must disconnect G
- (C)** No minimum spanning tree contains $e_{\max}$
- (D)** G has a unique minimum spanning tree

Q The number of distinct minimum spanning trees for the weighted graph below is ____ (**GATE-2017**) (2 Marks)



Q Let $G = (V, E)$ be any connected undirected edge-weighted graph. The weights of the edges in E are positive and distinct. Consider the following statements: **(Gate-2017) (2 Marks)**

I. Minimum Spanning Tree of G is always unique.

II. Shortest path between any two vertices of G is always unique.

Which of the above statements is/are necessarily true?

- a)** I only **b)** II only **c)** both I and II **d)** neither I and II

Q $G = (V, E)$ is an undirected simple graph in which each edge has a distinct weight, and e is a particular edge of G . Which of the following statements about the minimum spanning trees (MSTs) of G is/are TRUE **(Gate-2016) (2 Marks)**

- I.** If e is the lightest edge of some cycle in G , then every MST of G includes e
- II.** If e is the heaviest edge of some cycle in G , then every MST of G excludes e

(A) I only **(B)** II only **(C)** both I and II **(D)** neither I nor II

Q An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i and v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. The cost of the minimum spanning tree of such a graph with 10 nodes is: **(NET-NOV-2017)**

(a) 88

(b) 91

(c) 49

(d) 21

Q Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $4|i - j|$. The weight of minimum cost spanning tree of G is: **(NET-JULY-2016)**

(a) $4n^2$

(b) n

(c) $4n - 4$

(d) $2n - 2$

Q Let G be a weighted connected undirected graph with distinct positive edge weights. If every edge weight is increased by the same value, then which of the following statements is/are TRUE?
(Gate-2016) (2 Marks)

P: Minimum spanning tree of G does not change

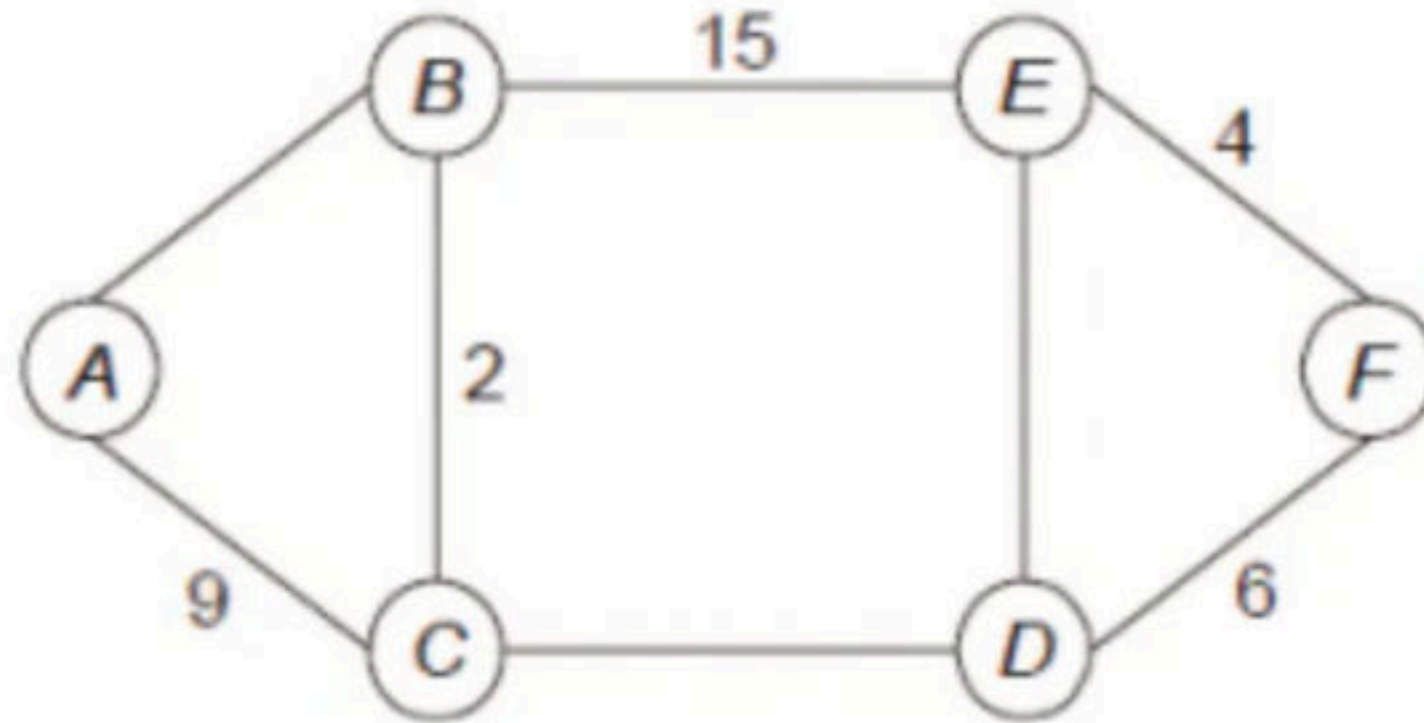
Q: Shortest path between any pair of vertices does not change

(A) P only **(B)** Q only **(C)** Neither P nor Q **(D)** Both P and Q

Q Let G be a complete undirected graph on 4 vertices, having 6 edges with weights being 1, 2, 3, 4, 5, and 6. The maximum possible weight that a minimum weight spanning tree of G can have is. **(Gate-2016) (2 Marks)**

Q Let G be connected undirected graph of 100 vertices and 300 edges. The weight of a minimum spanning tree of G is 500. When the weight of each edge of G is increased by five, the weight of a minimum spanning tree becomes _____. **(GATE-2015) (2 Marks)**

Q The graph shown below has 8 edges with distinct integer edge weights. The minimum spanning tree (MST) is of weight 36 and contains the edges: $\{(A, C), (B, C), (B, E), (E, F), (D, F)\}$. The edge weights of only those edges which are in the MST are given in the figure shown below. The minimum possible sum of weights of all 8 edges of this graph is _____. (**Gate - 2015**)
(2 Marks)



Q Let G be a weighted graph with edge weights greater than one and G' be the graph constructed by squaring the weights of edges in G . Let T and T' be the minimum spanning trees of G and G' , respectively, with total weights t and t' . Which of the following statements is TRUE? **(Gate-2012)**
(2 Marks)

(A) $T' = T$ with total weight $t' = t^2$

(C) $T' \neq T$ but total weight $t' = t^2$

(B) $T' = T$ with total weight $t' < t^2$

(D) None of the above

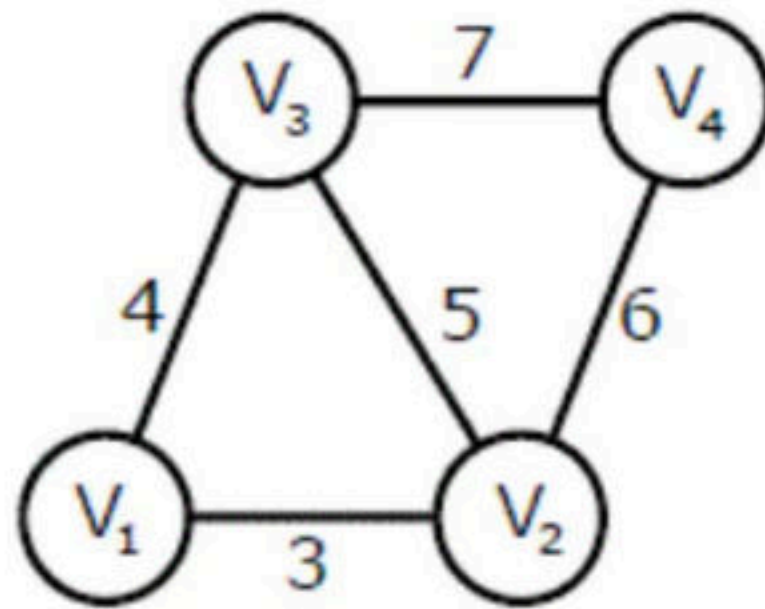
Q An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes? **(GATE - 2011) (2 Marks)**

(A) $\frac{1}{12}(11n^2 - 5n)$

(B) $n^2 - n + 1$

(C) $6n - 11$

(D) $2n + 1$



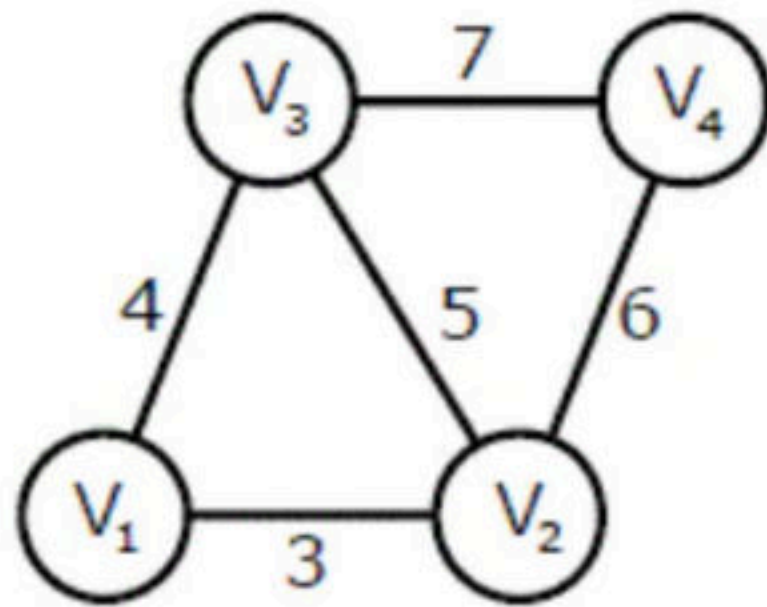
Q The length of the path from v_5 to v_6 in the MST of previous question with $n = 10$ is **(GATE - 2011) (2 Marks)**

(A) 11

(B) 25

(C) 31

(D) 41



Q Consider a complete undirected graph with vertex set $\{0, 1, 2, 3, 4\}$. Entry W_{ij} in the matrix W below is the weight of the edge $\{i, j\}$. What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T ? **(GATE - 2010) (2 Marks)**

(A) 7

(B) 8

(C) 9

(D) 10

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

Q In the graph given in above question, what is the minimum possible weight of a path P from vertex 1 to vertex 2 in this graph such that P contains at most 3 edges?

(GATE - 2010) (2 Marks)

(A) 7

(B) 8

(C) 9

(D) 10

Q Let w be the minimum weight among all edge weights in an undirected connected graph. Let e be a specific edge of weight w . Which of the following is FALSE? **(Gate-2007) (2 Marks)**

- (A)** There is a minimum spanning tree containing e .
- (B)** If e is not in a minimum spanning tree T , then in the cycle formed by adding e to T , all edges have the same weight.
- (C)** Every minimum spanning tree has an edge of weight w .
- (D)** e is present in every minimum spanning tree.

Q What is the largest integer m such that every simple connected graph with n vertices and n edges contains at least m different spanning trees? **(Gate-2007) (2 Marks)**

(A) 1

(B) 2

(C) 3

(D) n

Q Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. The weight of a minimum spanning tree of G is: **(GATE - 2006)**

(A) $n - 1$

(B) $2n - 2$

(C) nC_2

(D) 2

Q An undirected graph G has n nodes. Its adjacency matrix is given by an $n \times n$ square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. which one of the following is TRUE? **(Gate-2005) (2 Marks)**

- (A)** Graph G has no minimum spanning tree (MST)
- (B)** Graph G has a unique MST of cost $n-1$
- (C)** Graph G has multiple distinct MSTs, each of cost $n-1$
- (D)** Graph G has multiple spanning trees of different costs

Q Let s and t be two vertices in a undirected graph $G(V, E)$ having distinct positive edge weights. Let $[X, Y]$ be a partition of V such that $s \in X$ and $t \in Y$. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y , The edge e must definitely belong to: **(Gate-2005) (2 Marks)**

(A) the minimum weighted spanning tree of G

(B) the weighted shortest path from s to t

(C) each path from s to t

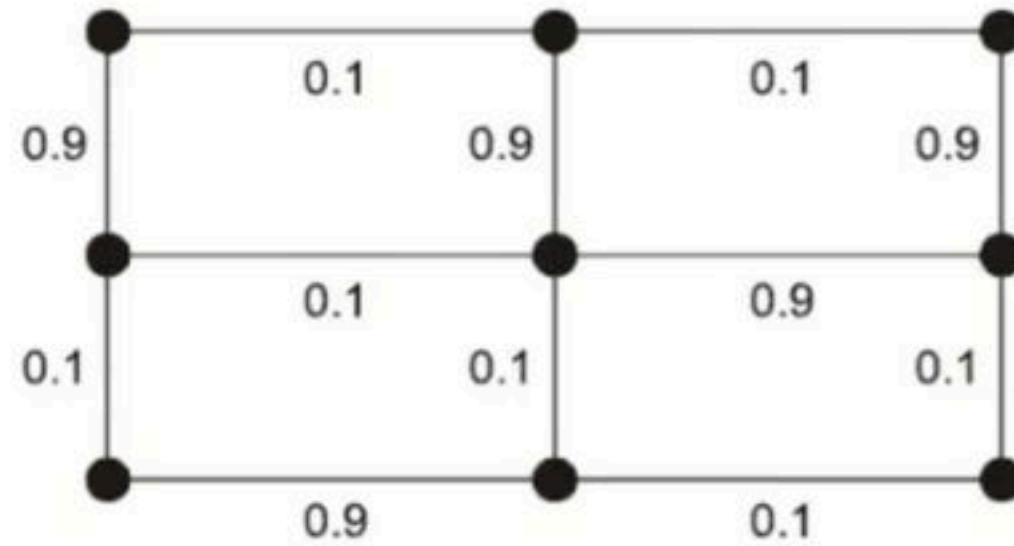
(D) the weighted longest path from s to t

Q Let G be a weighted undirected graph and e be an edge with maximum weight in G . Suppose there is a minimum weight spanning tree in G containing the edge e . Which of the following statements is always TRUE? **(Gate-2005)(2 Marks)**

- (A)** There exists a cut set in G having all edges of maximum weight.
- (B)** There exists a cycle in G having all edges of maximum weight
- (C)** Edge e cannot be contained in a cycle.
- (D)** All edges in G have the same weight

Q.38 Consider the following undirected graph with edge weights as shown:

(GATE-2021)



The number of minimum-weight spanning trees of the graph is _____.

Q.54 Let G be a connected undirected weighted graph. Consider the following two statements.

S_1 : There exists a minimum weight edge in G which is present in every minimum spanning tree of G .

S_2 : If every edge in G has distinct weight, then G has a unique minimum spanning tree.

Which one of the following options is correct?

(a) S_1 is false and S_2 is true.

(b) Both S_1 and S_2 are false.

(c) S_1 is true and S_2 is false.

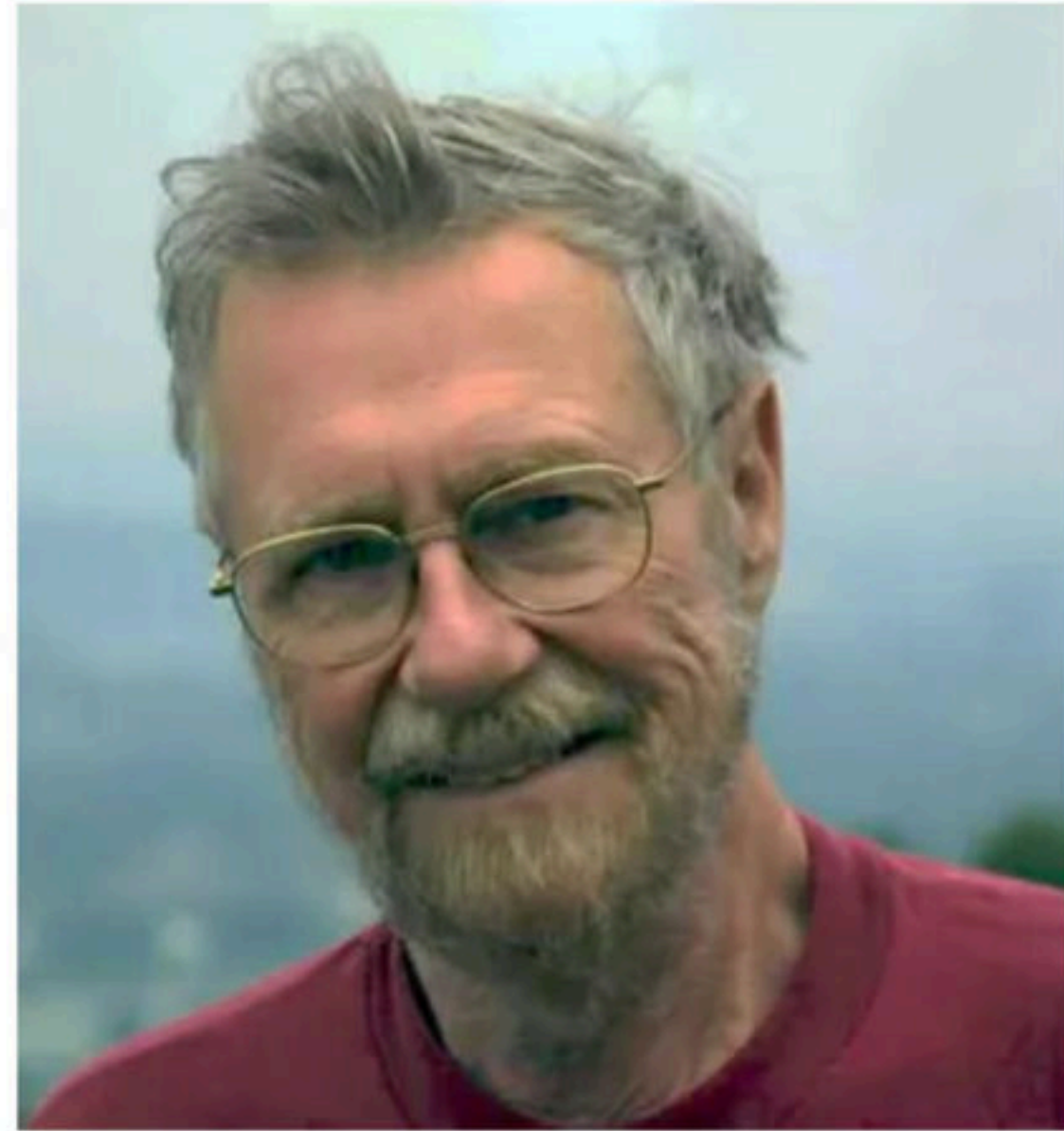
(d) Both S_1 and S_2 are true.

Single Source Shortest Path

- In graph theory, the **shortest path problem** is the problem of finding a path between two vertices (or nodes) in a graph such that the sum of the weights of its constituent edges is minimized.

Dijkstra's algorithm (or Dijkstra's Shortest Path First algorithm, SPF algorithm)

- Is an algorithm for finding the shortest paths between nodes in a graph, which may represent, for example, road networks. It was conceived by computer scientist Edsger W. Dijkstra in 1956 and published three years later.



- One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path.
- As I said, it was a twenty-minute invention. In fact, it was published in '59, three years later. The publication is still readable, it is, in fact, quite nice.

- One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities.
- Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame.
- A widely used application of shortest path algorithm is network routing protocols, most notably IS-IS (Intermediate System to Intermediate System) and Open Shortest Path First (OSPF).

- Dijkstra's algorithm uses a data structure for storing and querying partial solutions sorted by distance from the start. The original algorithm uses a min-priority queue and runs in time $O(V^2)$ (where V is the number of nodes)

Dijkstra algorithm (G, W, S)

```
{
    initialize-Single-source ( $G, S$ )
     $S \leftarrow \phi$ 
     $Q \leftarrow V[G]$ 
    While ( $Q \neq \phi$ )
    {
         $u \leftarrow \text{extract-min}(Q)$ 
         $S \leftarrow S \cup \{u\}$ 
        for each vertex  $v \in \text{adj}(u)$ 
        {
            relax ( $u, v, w$ )
        }
    }
}
```


Initialize_Single_Source (G, S)

{

 for each vertex $v \in V[G]$

 {

$d[v] \leftarrow \infty$

$\Pi[v] \leftarrow \text{NIL}$

 }

$d[S] \leftarrow 0$

}

Relax (u, v, w)

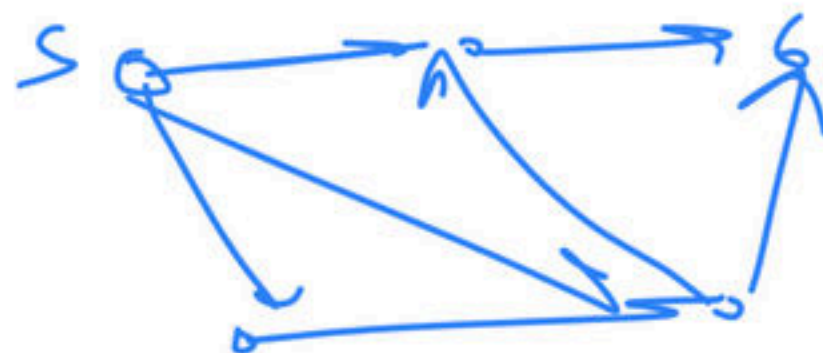
```
{  
    if( $d[v] > d[u] + w(u, v)$ )  
    {  
         $d[v] \leftarrow d[u] + w(u, v)$   
         $\Pi[v] \leftarrow u$   
    }  
}
```

- Guarantee to find optimal solution in a connected graph with positive weights.
- Can fail on graph with negative weights.

DIJKSTRA(G, w, s)

```
1 INITIALIZE-SINGLE-SOURCE( $G, s$ ) ✓  
2  $S = \emptyset$   
3  $Q = G.V$   
4 while  $Q \neq \emptyset$   
5      $u = \text{EXTRACT-MIN}(Q)$  ✓  
6      $S = S \cup \{u\}$   
7     for each vertex  $v \in G.Adj[u]$   
8         RELAX( $u, v, w$ ) ✓
```

a	b	c	d	e
0	5	7	20	∞



How fast is Dijkstra's algorithm? It maintains the min-priority queue Q by calling three priority-queue operations: INSERT (implicit in line 3), EXTRACT-MIN (line 5), and DECREASE-KEY (implicit in RELAX, which is called in line 8). The algorithm calls both INSERT and EXTRACT-MIN once per vertex. Because each vertex $u \in V$ is added to set S exactly once, each edge in the adjacency list $Adj[u]$ is examined in the **for** loop of lines 7–8 exactly once during the course of the algorithm. Since the total number of edges in all the adjacency lists is $|E|$, this **for** loop iterates a total of $|E|$ times, and thus the algorithm calls DECREASE-KEY at most $|E|$ times overall. (Observe once again that we are using aggregate analysis.)

DIJKSTRA(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 $S = \emptyset$

3 $Q = G.V$ — $O(V)$

4 while $Q \neq \emptyset$ — $O(V)$

5 $u = \text{EXTRACT-MIN}(Q)$ — $O(V)$

6 $S = S \cup \{u\}$

7 for each vertex $v \in G.\text{Adj}[u]$ — $O(E)$

8 RELAX(u, v, w)

array

.	.	.	.	.
-	-	-	-	-

The running time of Dijkstra's algorithm depends on how we implement the min-priority queue. Consider first the case in which we maintain the min-priority queue by taking advantage of the vertices being numbered 1 to $|V|$. We simply store $v.d$ in the v th entry of an array. Each INSERT and DECREASE-KEY operation takes $O(1)$ time, and each EXTRACT-MIN operation takes $O(V)$ time (since we have to search through the entire array), for a total time of $O(V^2 + E) = O(V^2)$.

$$O(V^2 + E)$$

DIJKSTRA(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 $S = \emptyset$

3 $Q = G.V$

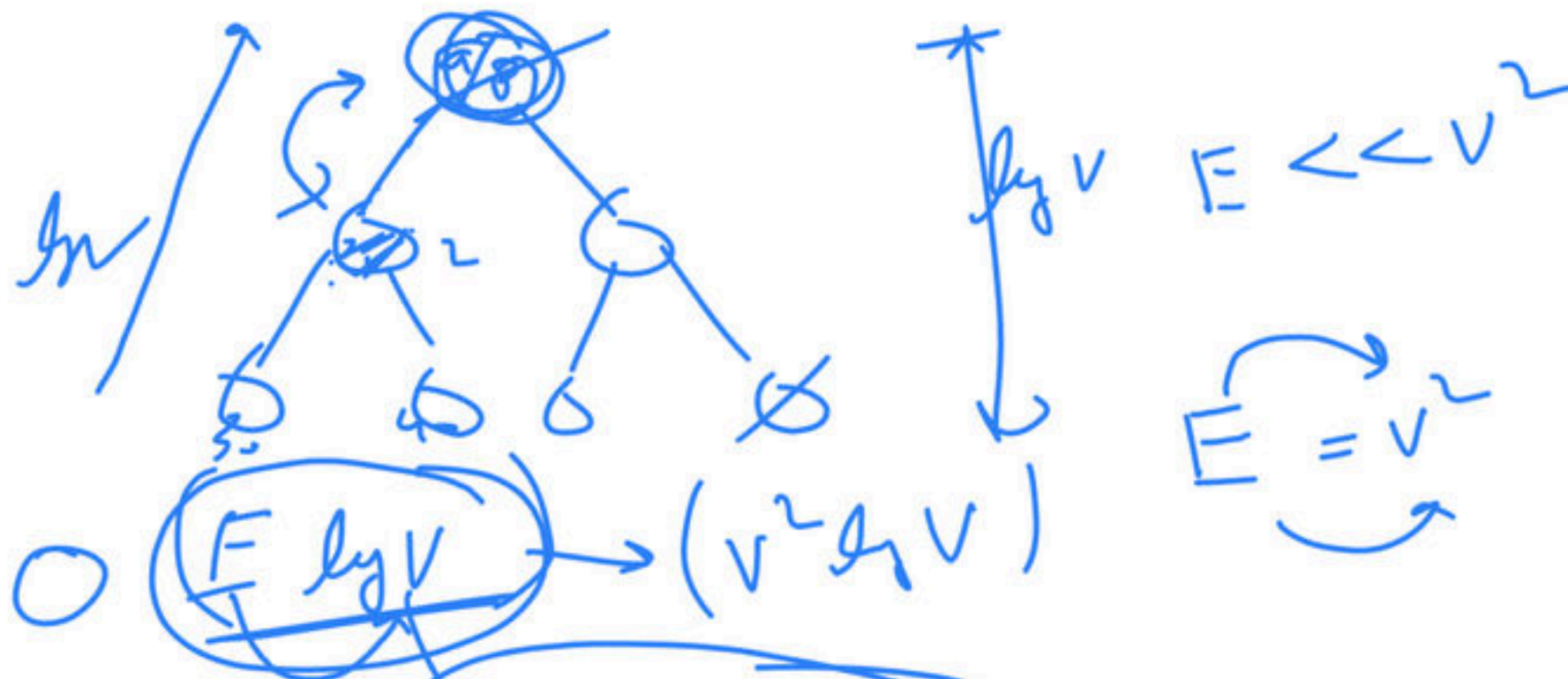
4 **while** $Q \neq \emptyset$

5 $u = \text{EXTRACT-MIN}(Q)$

6 $S = S \cup \{u\}$

7 **for each** vertex $v \in G.\text{Adj}[u]$

8 RELAX(u, v, w)



If the graph is sufficiently sparse—in particular, $E = o(V^2 / \lg V)$ —we can improve the algorithm by implementing the min-priority queue with a binary min-heap. (As discussed in Section 6.5, the implementation should make sure that vertices and corresponding heap elements maintain handles to each other.) Each EXTRACT-MIN operation then takes time $O(\lg V)$. As before, there are $|V|$ such operations. The time to build the binary min-heap is $O(V)$. Each DECREASE KEY operation takes time $O(\lg V)$, and there are still at most $|E|$ such operations. The total running time is therefore $O((V + E) \lg V)$, which is $O(E \lg V)$ if all vertices are reachable from the source. This running time improves upon the straightforward $O(V^2)$ -time implementation if $E = o(V^2 / \lg V)$.

DIJKSTRA(G, w, s)

```
1 INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  $S = \emptyset$ 
3  $Q = G.V$ 
4 while  $Q \neq \emptyset$ 
5      $u = \text{EXTRACT-MIN}(Q)$ 
6      $S = S \cup \{u\}$ 
7     for each vertex  $v \in G.Adj[u]$ 
8         RELAX( $u, v, w$ )
```

We can in fact achieve a running time of $O(V \lg V + E)$ by implementing the min-priority queue with a Fibonacci heap (see Chapter 19). The amortized cost of each of the $|V|$ EXTRACT-MIN operations is $O(\lg V)$, and each DECREASE-KEY call, of which there are at most $|E|$, takes only $O(1)$ amortized time. Historically, the development of Fibonacci heaps was motivated by the observation that Dijkstra's algorithm typically makes many more DECREASE-KEY calls than EXTRACT-MIN calls, so that any method of reducing the amortized time of each DECREASE-KEY operation to $o(\lg V)$ without increasing the amortized time of EXTRACT-MIN would yield an asymptotically faster implementation than with binary heaps.

Q Which of the following algorithms solves the single-source shortest paths? (**NET-JULY-2018**)

- (a) Prim's algorithm
- (b) Floyd - Warshall algorithm
- (c) Johnson's algorithm
- (d) Dijkstra's algorithm

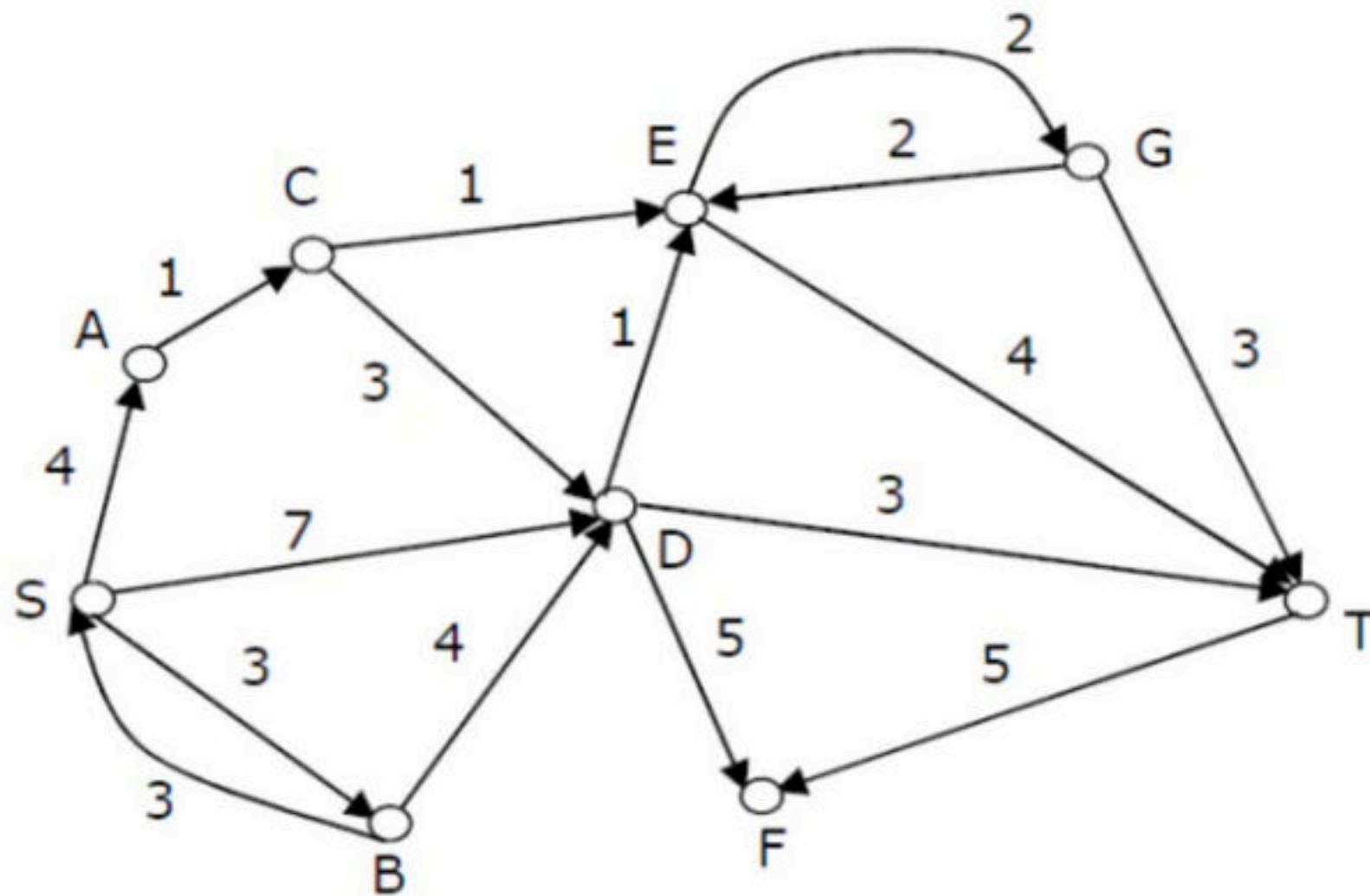
Q Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices S and T. Which one will be reported by Dijkstra's shortest path algorithm? Assume that, in any iteration, the shortest path to a **(Gate-2012) (2 Marks)**

a) SDT

b) SBDT

c) SACDT

d) SACET



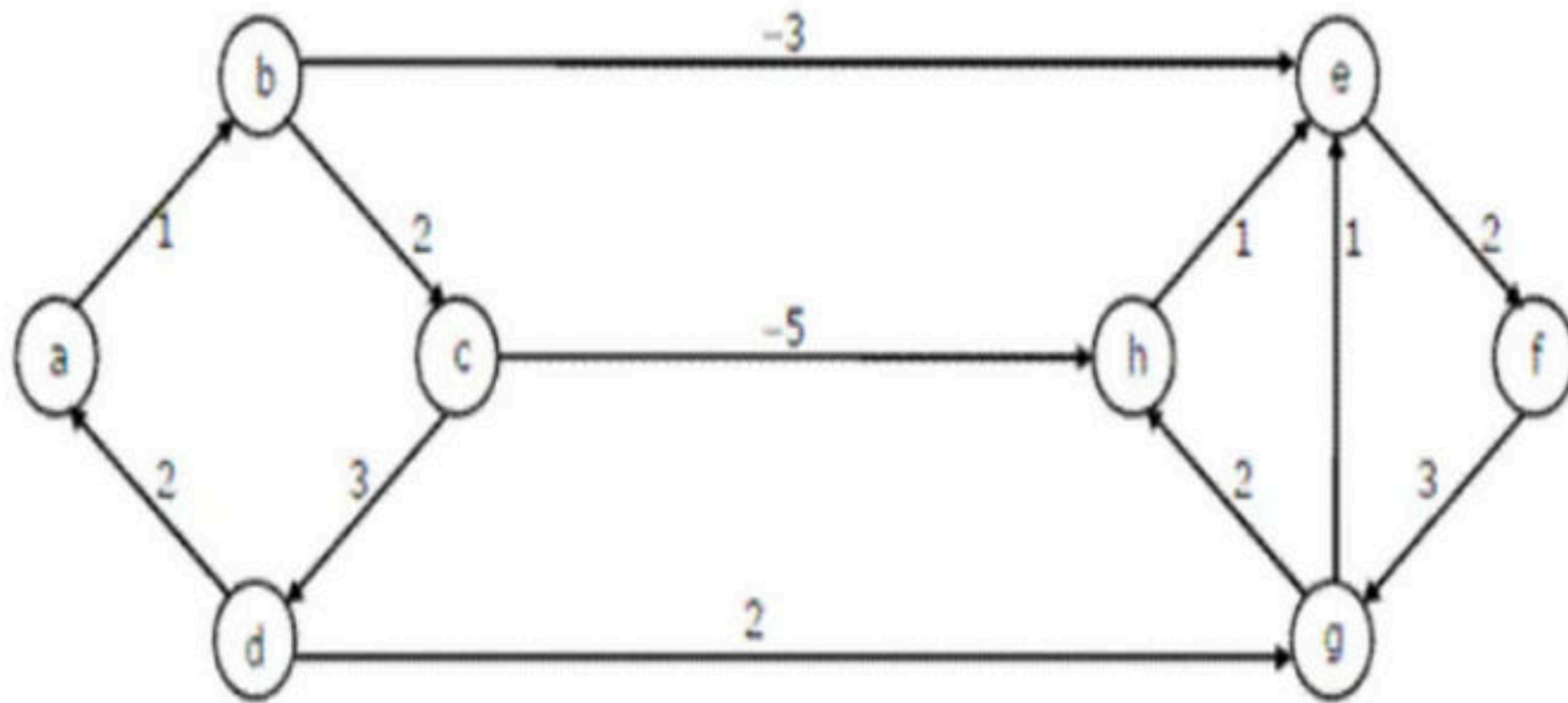
Q Dijkstra's single source shortest path algorithm when run from vertex a in the below graph, computes the correct shortest path distance to **(Gate-2008) (2 Marks)**

(A) only vertex a

(C) only vertices a, b, c, d

(B) only vertices a, e, f, g, h

(D) all the vertices



Q To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is: **(Gate-2006) (1 Marks)**

(A) Queue

(B) Stack

(C) Heap

(D) B-Tree

Q Let $G(V, E)$ an undirected graph with positive edge weights. Dijkstra's single-source shortest path algorithm can be implemented using the binary heap data structure with time complexity: **(Gate-2005) (2 Marks)**

(A) $O(|V|^2)$

(B) $O(|E| + |V| \log |V|)$

(C) $O(|V| \log |V|)$

(D) $O((|E| + |V|) \log |V|)$

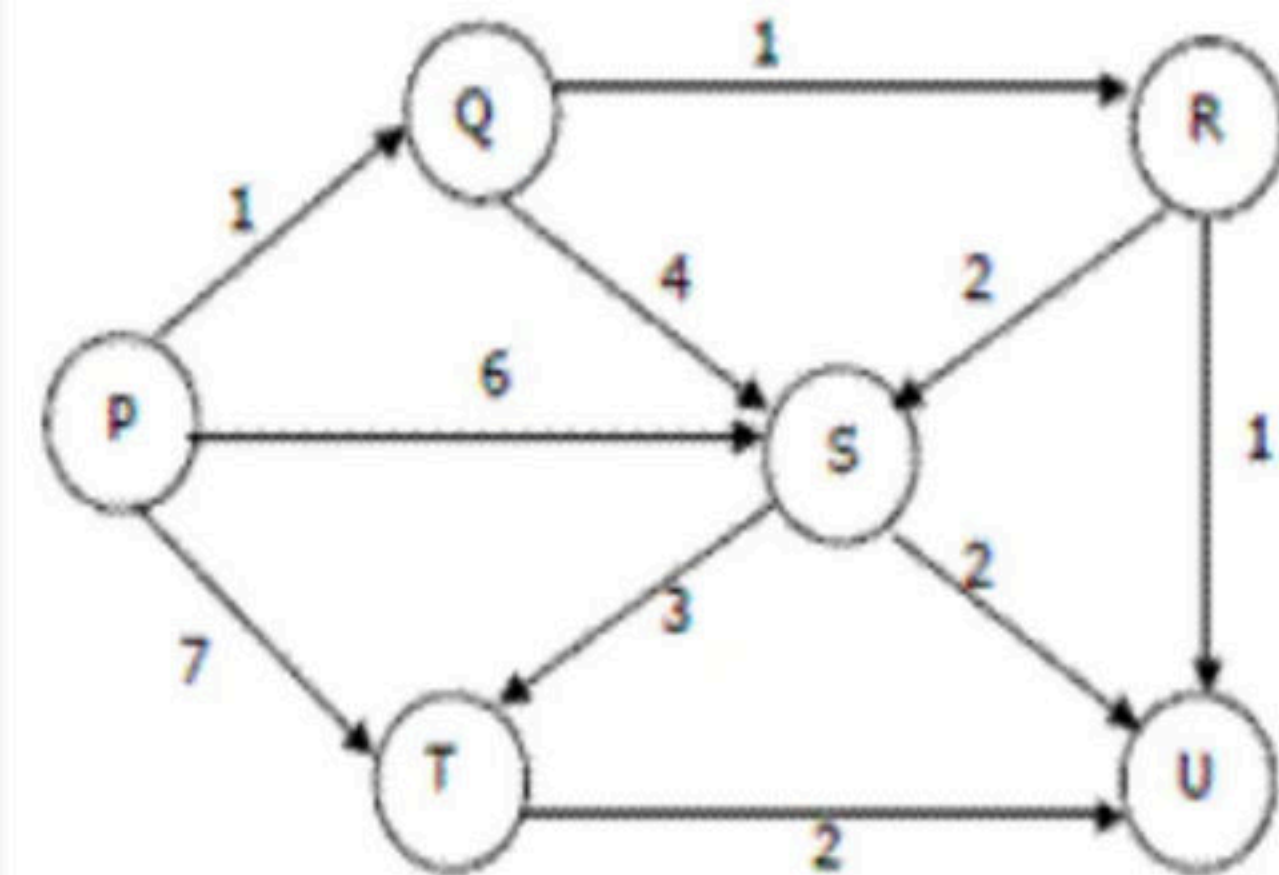
Q Suppose we run Dijkstra's single source shortest-path algorithm on the following edge weighted directed graph with vertex P as the source. In what order do the nodes get included into the set of vertices for which the shortest path distances are finalized? **(GATE - 2004) (2 Marks)**

(A) P, Q, R, S, T, U

(C) P, Q, R, U, T, S

(B) P, Q, R, U, S, T

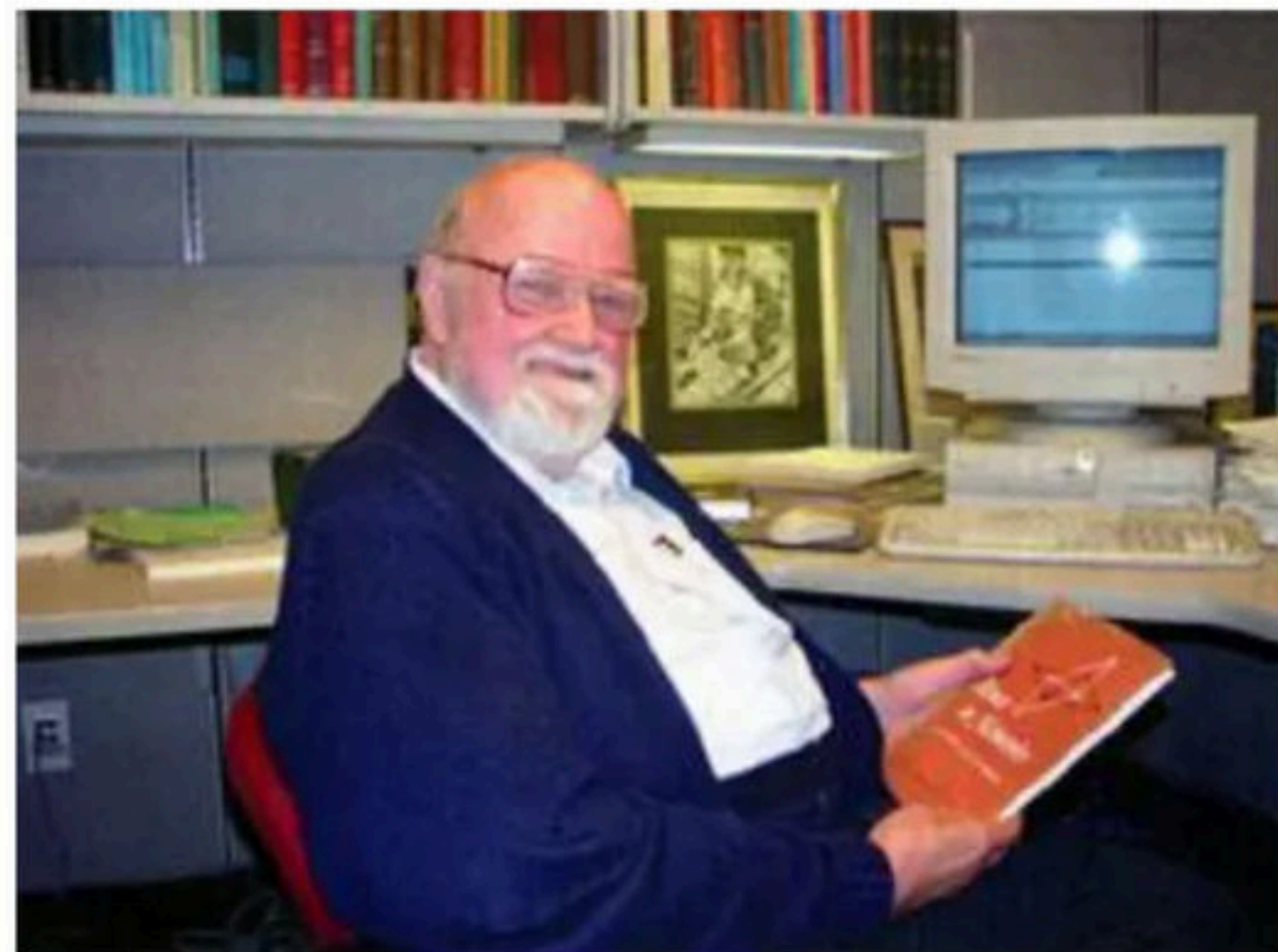
(D) P, Q, T, R, U, S



Break



Richard E. Bellman



L. R. Ford Jr

- The **Bellman–Ford algorithm** is an algorithm that computes shortest paths from a single source vertex to all of the other vertices in a weighted digraph.
- It is slower than Dijkstra's algorithm for the same problem, but more versatile, as it is capable of handling graphs in which some of the edge weights are negative numbers.
- The algorithm was first proposed by Alfonso Shimbel (1955), but is instead named after Richard Bellman and Lester Ford Jr., who published it in 1958 and 1956, respectively.


```
Bellman_ford (G, W, S)
{
    initialize-Single-Source (G, S)
    for  $i \leftarrow 1$  to  $|V(G)| - 1$ 
    {
        for each edge  $(u, v) \in E(G)$ 
        {
            Relax( $u, v, w$ )
        }
    }
    for each edge  $(u, v) \in E(G)$ 
    {
        if( $d[v] > d[u] + w(u, v)$ )
        {
            Return false
        }
    }
}
```

Initialize_Single_Source (G, S)

{

 for each vertex $v \in V[G]$

 {

$d[v] \leftarrow \infty$

$\Pi[v] \leftarrow \text{NIL}$

 }

$d[S] \leftarrow 0$

}

Relax (u, v, w)

{

if($d[v] \leftarrow d[u] + w(u, v)$)

{

$d[v] \leftarrow d[u] + w(u, v)$

$\pi[v] \leftarrow u$

}

}

BELLMAN-FORD(G, w, s)

```
1 INITIALIZE-SINGLE-SOURCE( $G, s$ )  $O(V)$ 
2 for  $i = 1$  to  $|G.V| - 1$   $O(V)$ 
3   for each edge  $(u, v) \in G.E$   $E$ 
4     RELAX( $u, v, w$ )  $O(V E)$ 
5   for each edge  $(u, v) \in G.E$   $O(E)$ 
6     if  $v.d > u.d + w(u, v)$ 
7       return FALSE
8 return TRUE
```

$$\underline{O(V E)} \rightarrow O(V^2)$$

$$V^2 = E$$

$$E \ll V^2$$

$$\cancel{O(V E) + E}$$

The Bellman-Ford algorithm runs in time $O(VE)$, since the initialization in line 1 takes $\Theta(V)$ time, each of the $|V| - 1$ passes over the edges in lines 2–4 takes $\Theta(E)$ time, and the **for** loop of lines 5–7 takes $O(E)$ time.

Q An all-pairs shortest-paths problem is efficiently solved using: **(NET-JUNE-2018)**

- a)** Dijkstra' algorithm
- c)** Kruskal algorithm

- b)** Bellman-Ford algorithm
- d)** Floyd-Warshall algorithm

Q Consider the following table

Algorithms	Design Paradigms
(P) Kruskal	(i) Divide and Conquer
(Q) Quicksort	(ii) Greedy
(R) Floyd-Warshall	(iii) Dynamic Programming

Match the algorithm to design paradigms they are based on: **(Gate-2017) (2 Marks)**

a) P-(ii), Q-(iii), R-(i)

b) P-(iii), Q-(i), R-(ii)

c) P-(ii), Q-(i), R-(iii)

d) P-(i), Q-(ii), R-(iii)

Q Match the following (Gate-2015) (2 Marks)

List-I	List-II
A. Prim's algorithm for minimum spanning tree	1. Backtracking
B. Floyd-Warshall algorithm for all pairs shortest paths	2. Greedy method
C. Merge sort	3. Dynamic programming
D. Hamiltonian circuit	4. Divide and conquer

	A	B	C	D
a)	3	2	4	1
b)	1	2	4	3
c)	2	3	4	1
d)	2	1	3	4

Q What is the time complexity of Bellman-Ford single-source shortest path algorithm on a complete graph of n vertices? **(Gate-2013) (1 Marks)**

a) $\Theta(n^3)$

b) $\Theta(n^2)$

c) $\Theta(n^2 \log n)$

d) $\Theta(n^3 \log n)$

Q Consider a weighted undirected graph with positive edge weights and let (u, v) be an edge in the graph. It is known that the shortest path from source vertex s to u has weight 53 and shortest path from s to v has weight 65. Which statement is always true? **(NET-JUNE-2012)**

(A) Weight $(u, v) < 12$

(B) Weight $(u, v) = 12$

(C) Weight $(u, v) > 12$

(D) Weight $(u, v) \geq 12$

Q Which of the following statement(s) is/are correct regarding Bellman-Ford shortest path algorithm? **(Gate-2009) (1 Marks)**

P: Always finds a negative weighted cycle, if one exists.

Q: Finds whether any negative weighted cycle is reachable from the source.

- a)** P only **b)** Q only **c)** Both P and Q **d)** Neither P nor Q

Q In an unweighted, undirected connected graph, the shortest path from a node S to every other node is computed most efficiently, in terms of time complexity by **(Gate-2007) (2 Marks)**

(A) Dijkstra's algorithm starting from S.

(B) Warshall's algorithm

(C) Performing a DFS starting from S.

(D) Performing a BFS starting from S.

Break

Q Is the following statement valid?

Given a weighted graph where weights of all edges are unique (no two edge have same weights), there is always a unique shortest path from a source to destination in such a graph.

(A) True

(B) False

Q Is the following statement valid?

Given a graph where all edges have positive weights, the shortest paths produced by Dijkstra and Bellman Ford algorithm may be different but path weight would always be same.

(A) True

(B) False

Q Is the following statement valid about shortest paths?

Given a graph, suppose we have calculated shortest path from a source to all other vertices. If we modify the graph such that weights of all edges is becomes double of the original weight, then the shortest path remains same only the total weight of path changes.

(A) True

(B) False



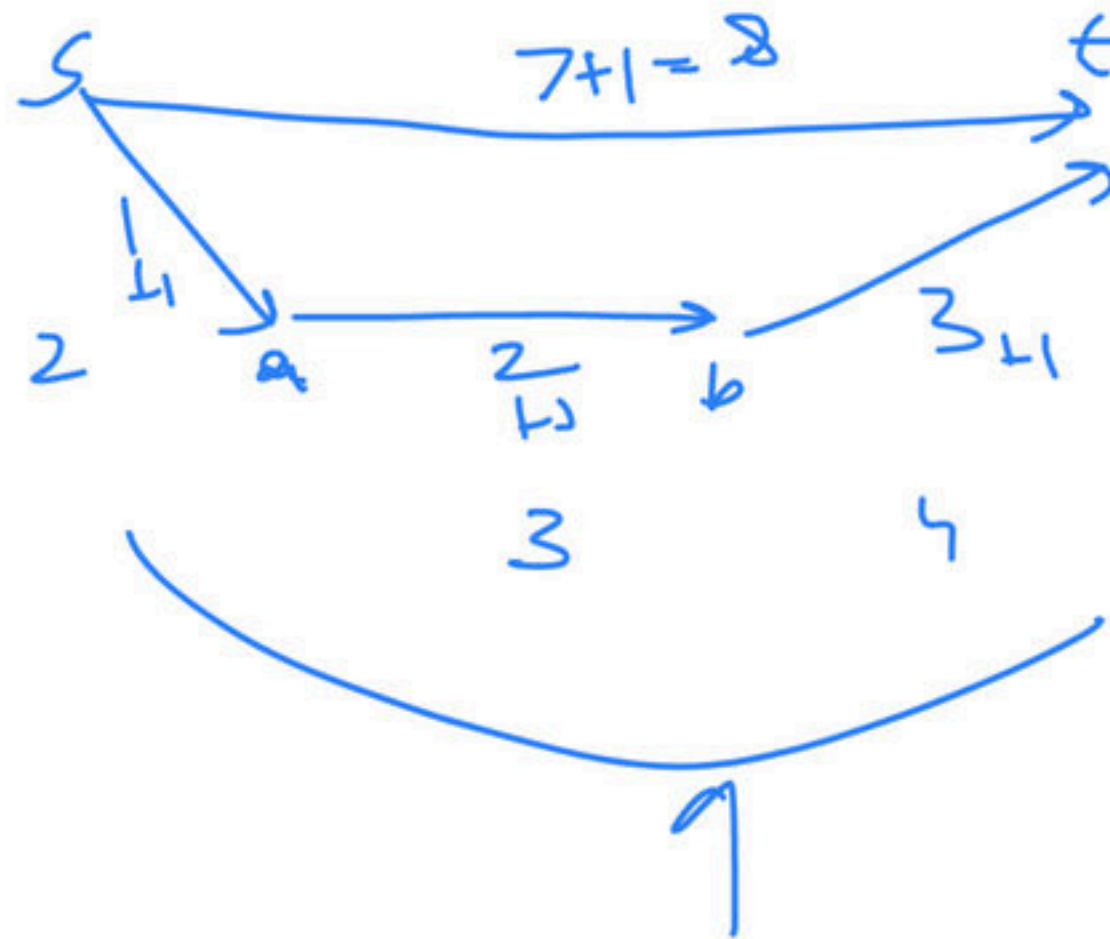
Q (In a weighted graph, assume that the shortest path from a source 's' to a destination 't' is correctly calculated using a shortest path algorithm.) Is the following statement true?
If we increase weight of every edge by 1, the shortest path always remains same.

(A) Yes

32

~~(B) No~~

18



Q Which of the following standard algorithms is not a Greedy algorithm?

(A) Dijkstra's shortest path algorithm

(B) Prim's algorithm

(C) Kruskal algorithm

(D) Huffman Coding

(E) Bellman Ford Shortest path algorithm

Q Which of the following standard algorithms is not Dynamic Programming based.

(A) Bellman–Ford Algorithm for single source shortest path

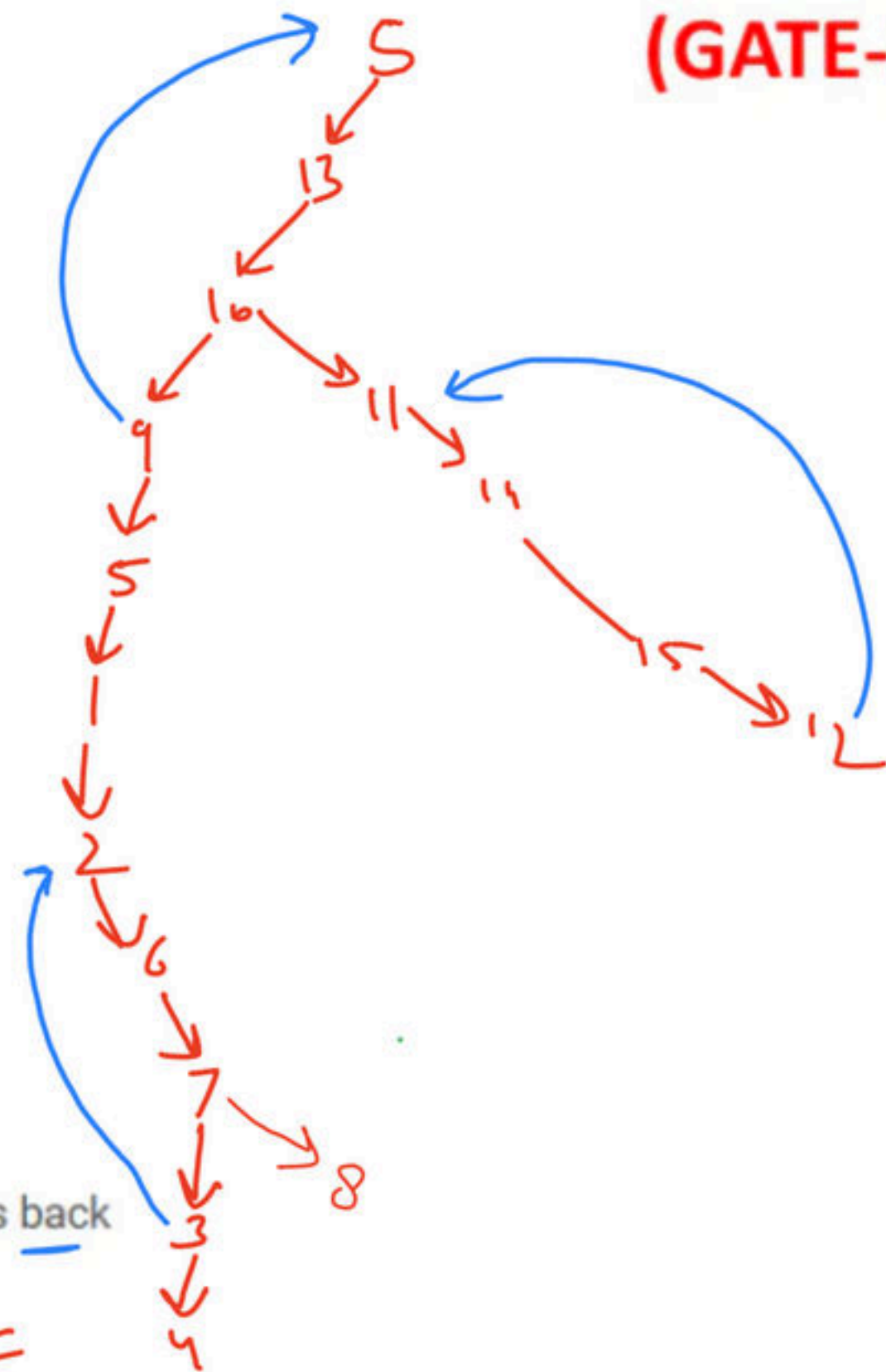
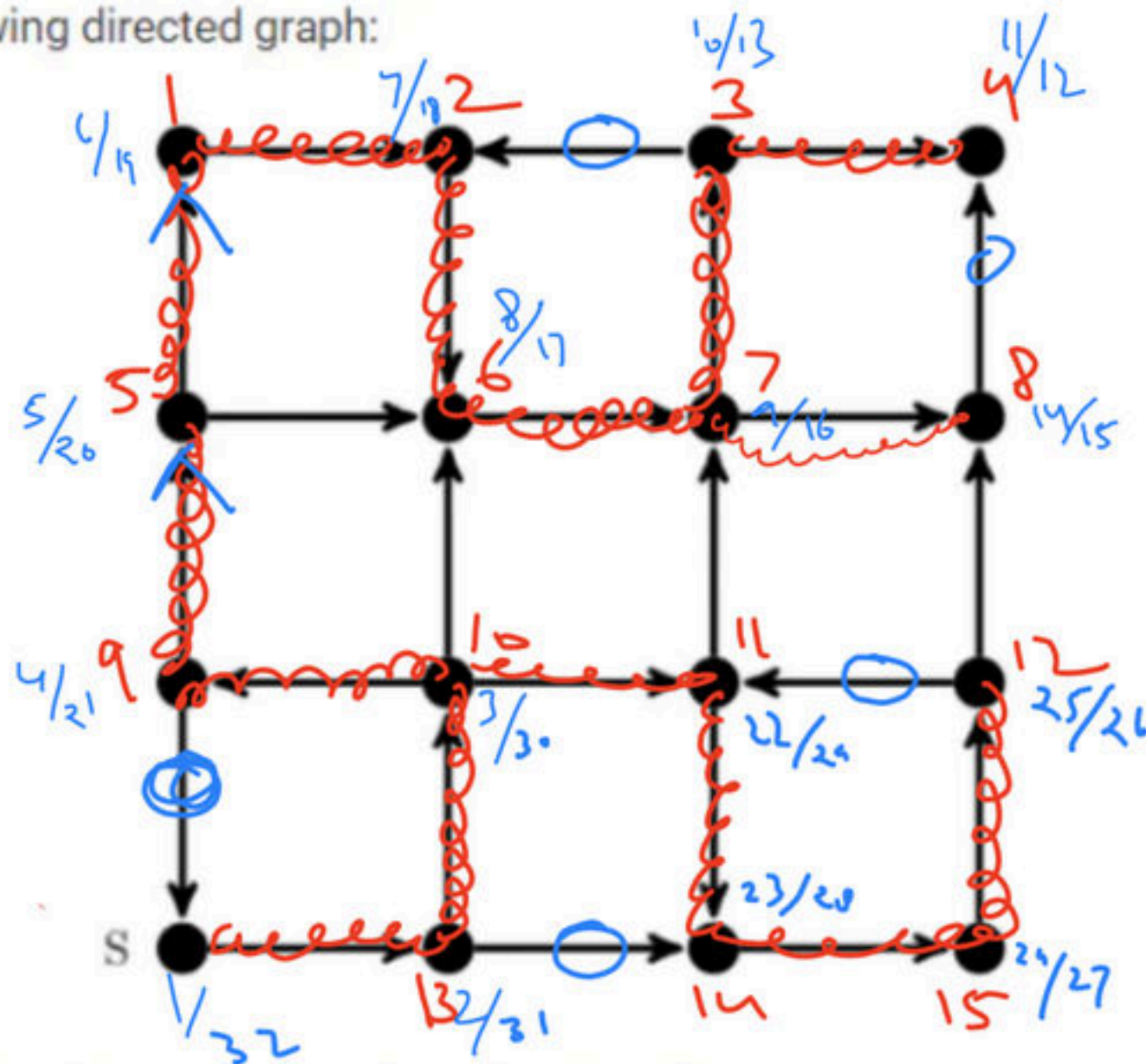
(B) Floyd Warshall Algorithm for all pairs shortest paths

(C) 0-1 Knapsack problem

(D) Prim's Minimum Spanning Tree

(GATE-2021)

Consider the following directed graph:



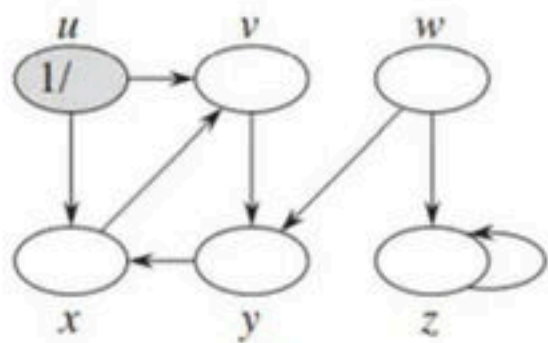
Which of the following is/are correct about the graph?

- A. The graph does not have a topological order — T
- B. A depth-first traversal starting at vertex S classifies three directed edges as back edges — T
- C. The graph does not have a strongly connected component — F
- D. For each pair of vertices u and v , there is a directed path from u to v — F

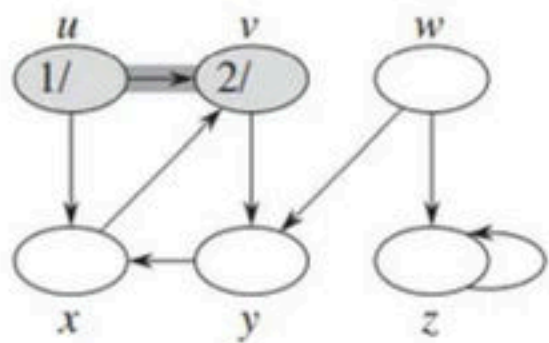
We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

1. **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
2. **Back edges** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.

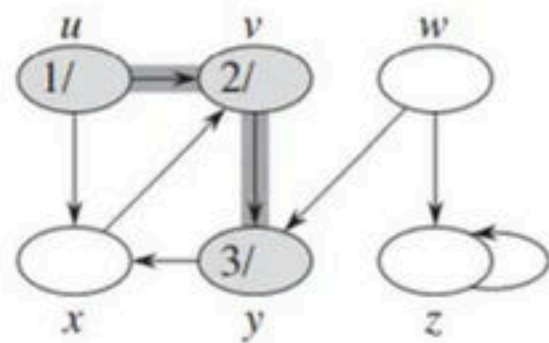
3. **Forward edges** are those nontree edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.
4. **Cross edges** are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.



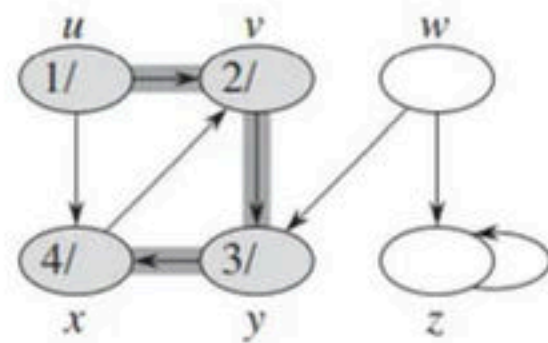
(a)



(b)

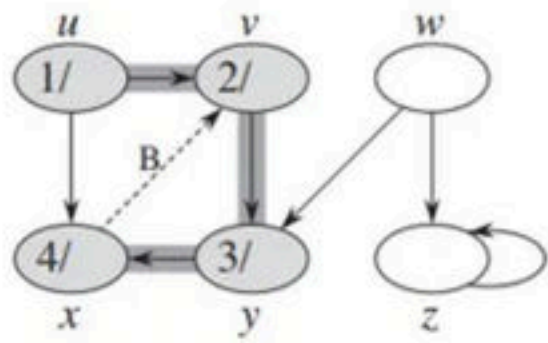


(c)

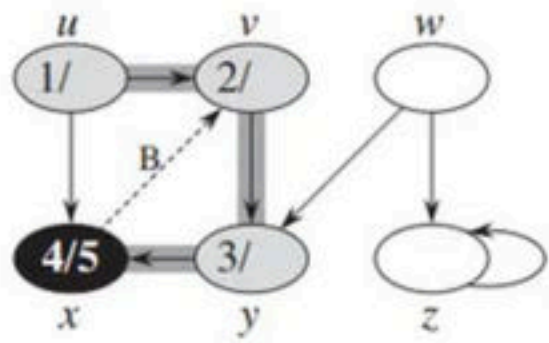


(d)

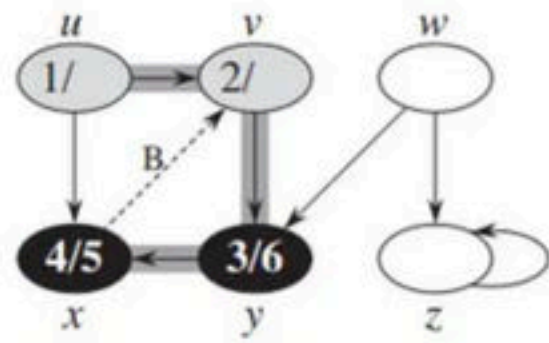
9/6



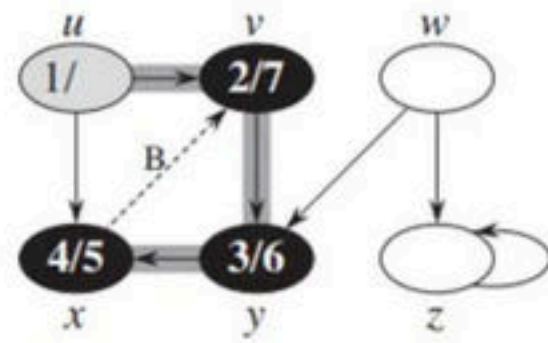
(e)



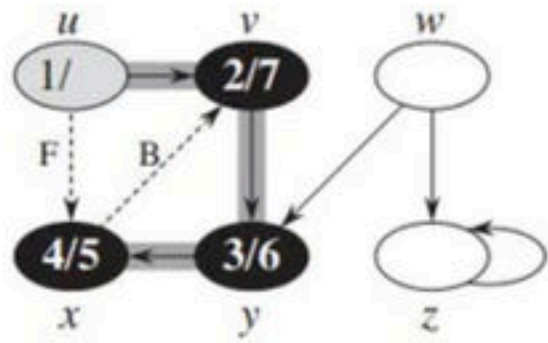
(f)



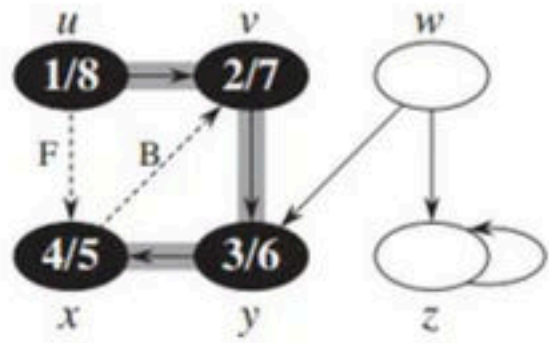
(g)



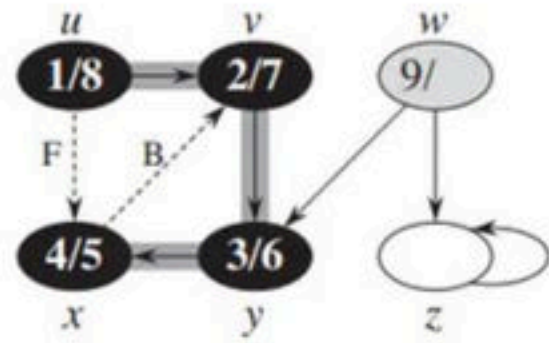
(h)



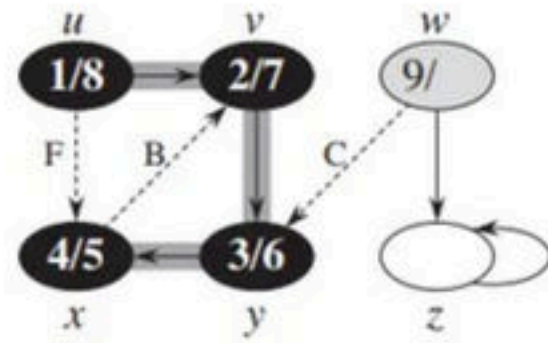
(i)



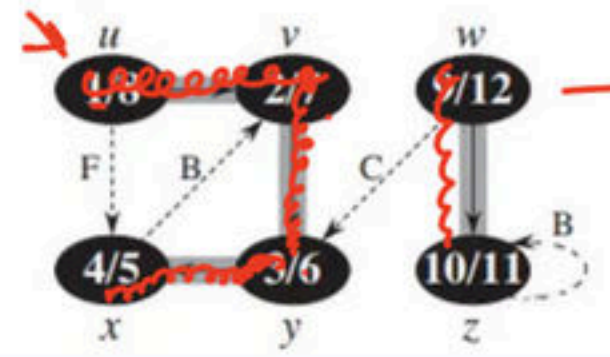
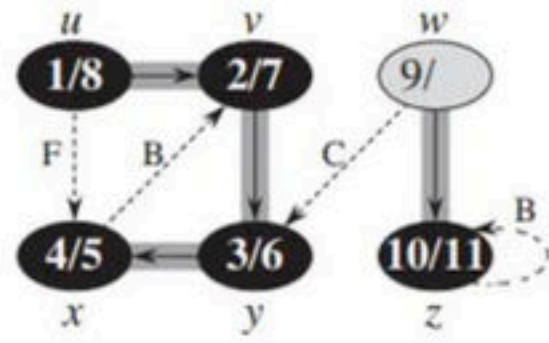
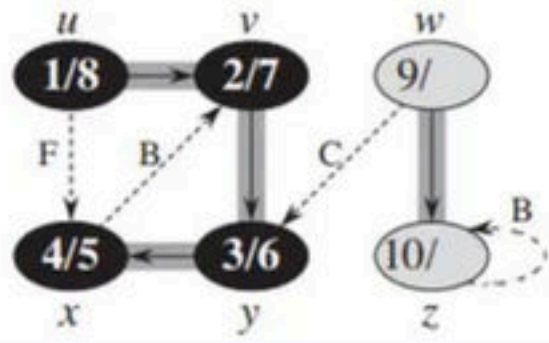
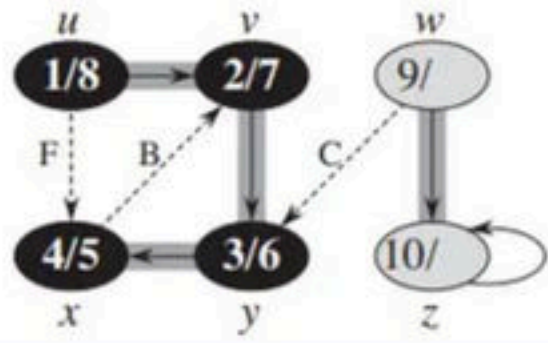
(j)



(k)



(l)

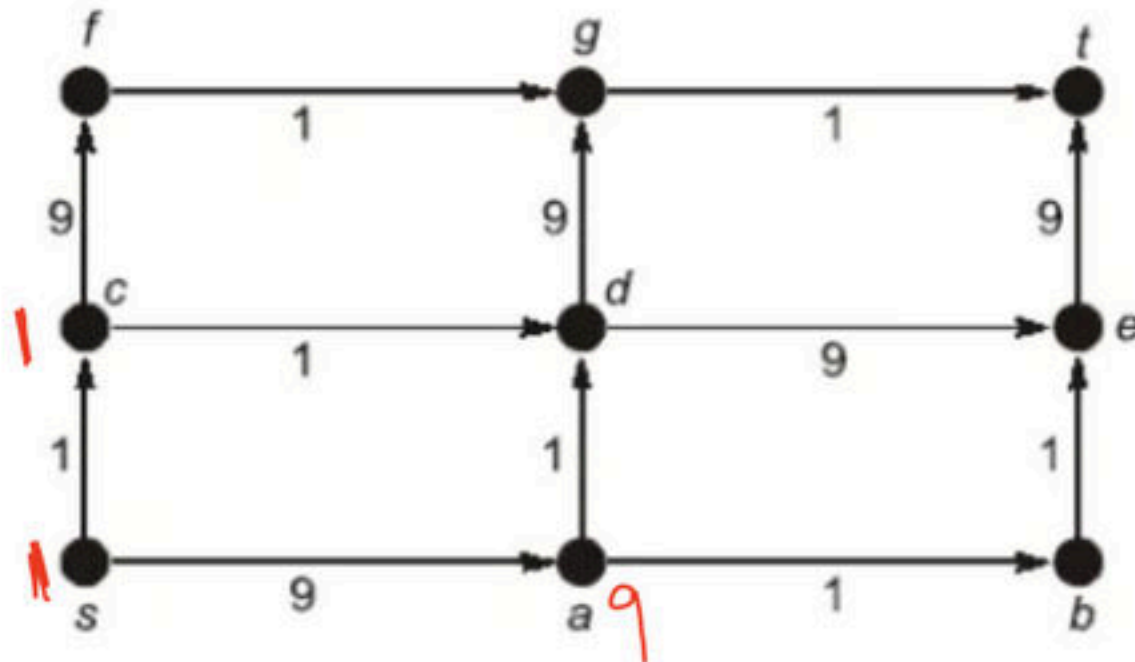


(GATE-2021)

929

Q.9

In a directed acyclic graph with a source vertex s , the quality-score of a directed path is defined to be the product of the weights of the edges on the path. Further, for a vertex v other than s , the quality-score of v is defined to be the maximum among the quality-scores of all the paths from s to v . The quality-score of s is assumed to be 1.



The sum of the quality-scores of all the vertices in the graph shown above is _____.

929

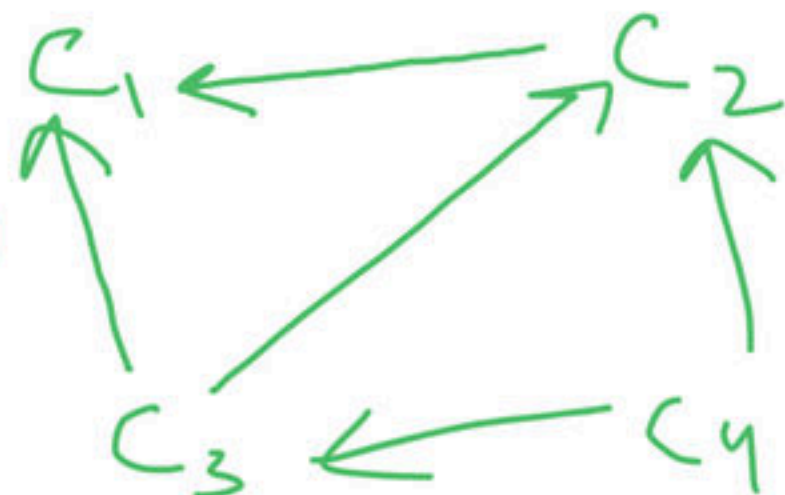
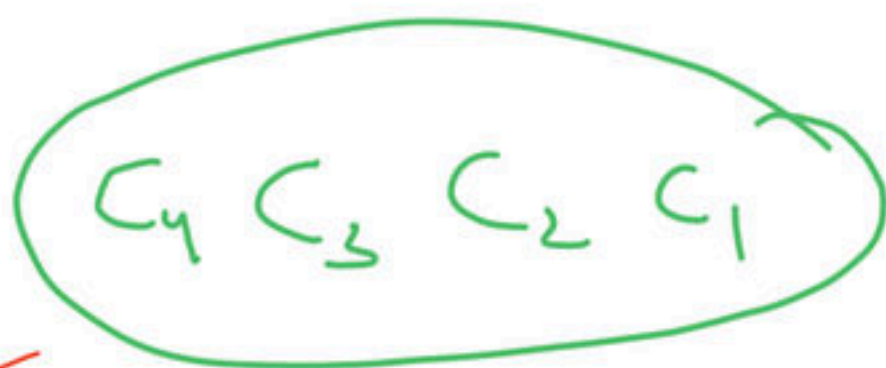
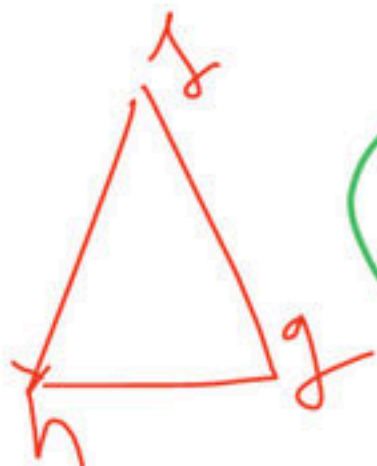
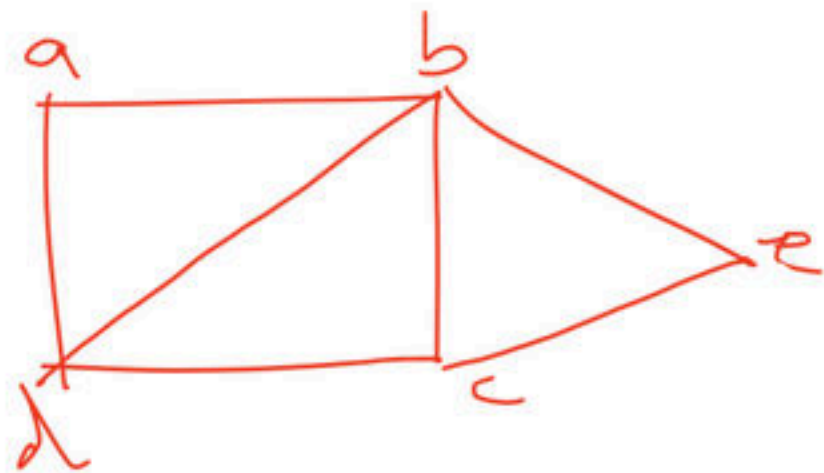
DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

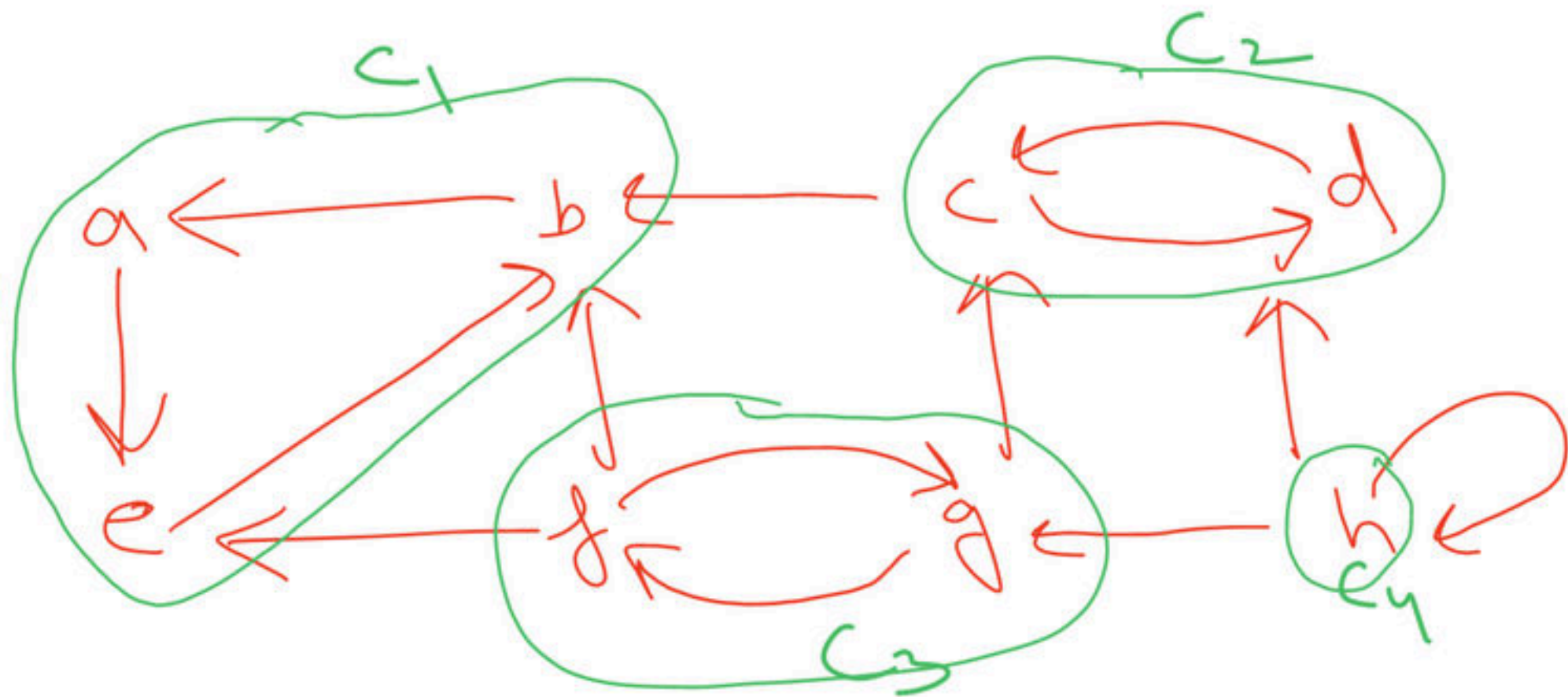
```
1   $time = time + 1$                                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$                             // explore edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$                                 // blacken  $u$ ; it is finished
9   $time = time + 1$ 
10  $u.f = time$ 
```

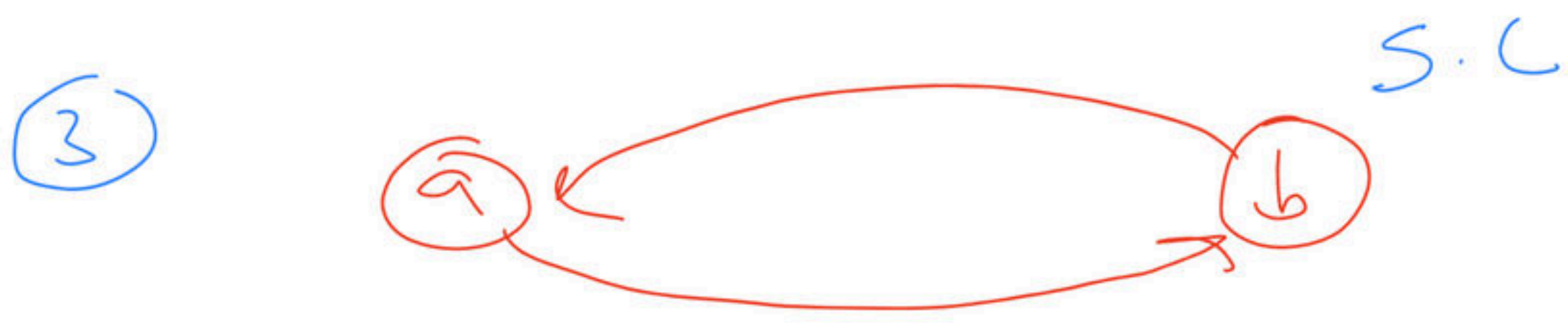
①

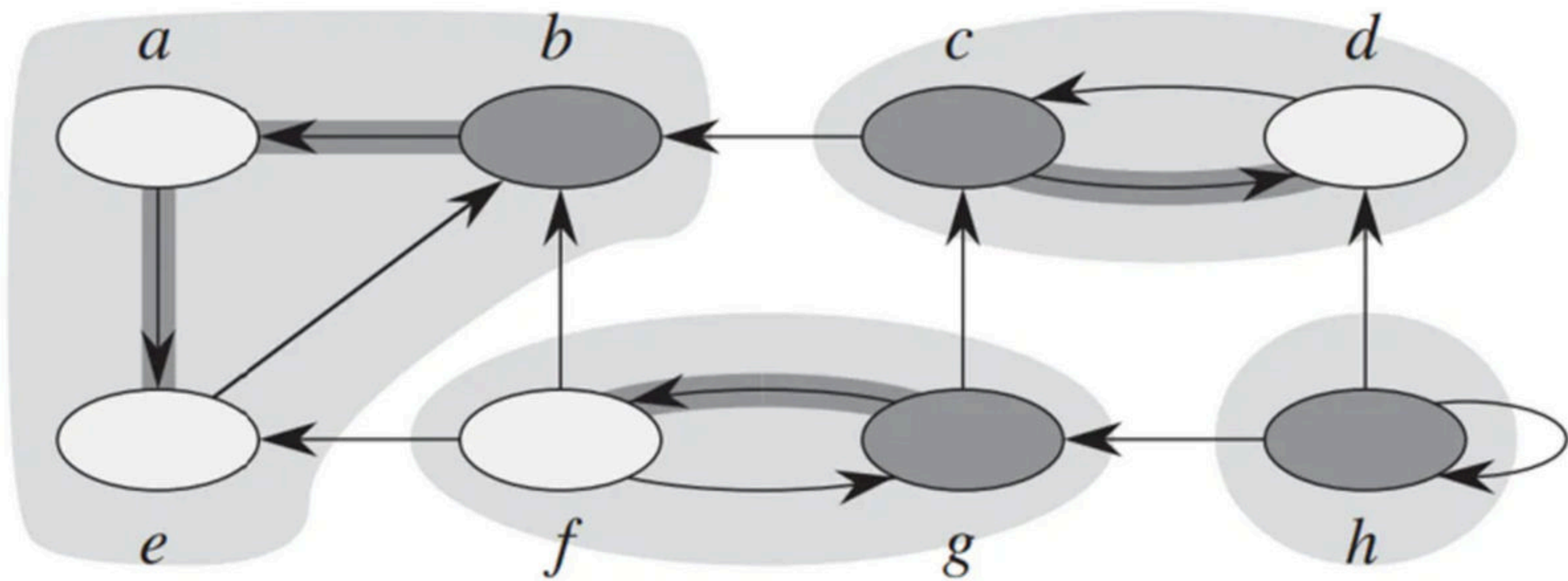


DAG

②







Q.25 Let $G = (V, E)$ be an undirected unweighted connected graph. The diameter of G is defined as

$$\text{diam}(G) = \max_{u, v \in V} \{\text{the length of shortest path between } u \text{ and } v\}$$

Let M be the adjacency matrix of G .

Define graph G_2 on the same set of vertices with adjacency matrix N , where

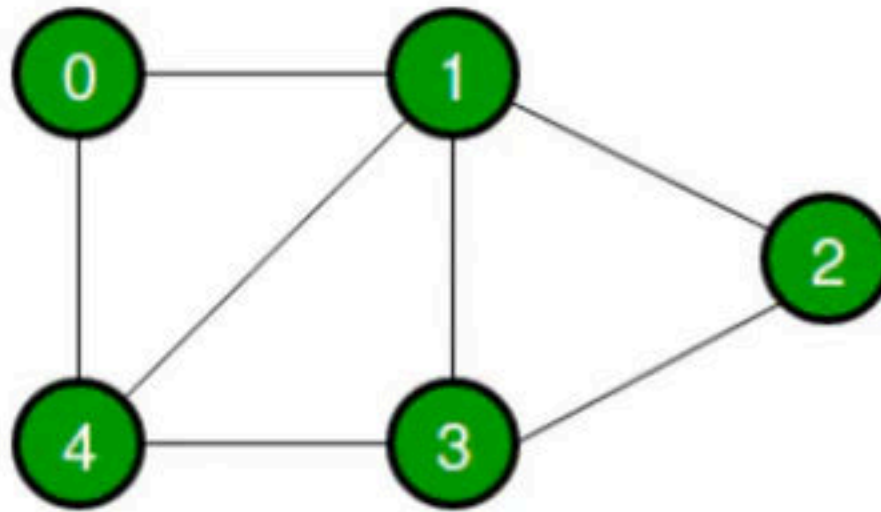
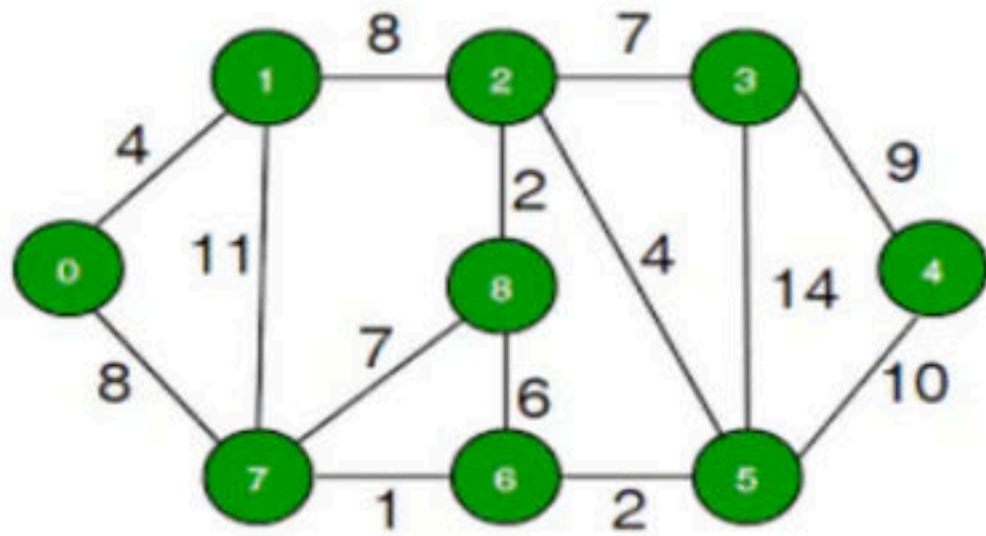
$$N_{ij} = \begin{cases} 1 & \text{if } M_{ij} > 0 \text{ or } P_{ij} > 0, \text{ where } P = M^2 \\ 0 & \text{Otherwise} \end{cases}$$

Which one of the following statements is true?

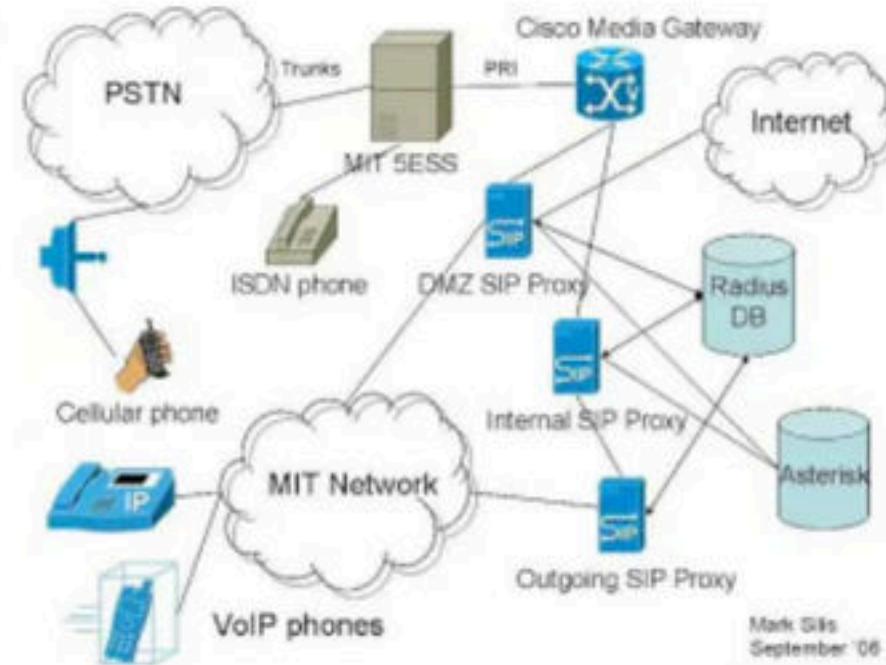
- (a) $\lceil \text{diam}(G)/2 \rceil < \text{diam}(G_2) < \text{diam}(G)$
- (b) $\text{diam}(G_2) \leq \lceil \text{diam}(G)/2 \rceil$
- (c) $\text{diam}(G) < \text{diam}(G_2) \leq 2 \text{ diam}(G)$
- (d) $\text{diam}(G_2) = \text{diam}(G)$

Graph

- Graph is a data structure that consists of following two components:
 - A finite set of vertices also called as nodes.
 - A finite set of ordered pair of the form (u, v) called as edge. The pair is ordered because (u, v) is not same as (v, u) in case of a directed graph(di-graph).
 - The pair of the form (u, v) indicates that there is an edge from vertex u to vertex v . The edges may contain weight/value/cost.



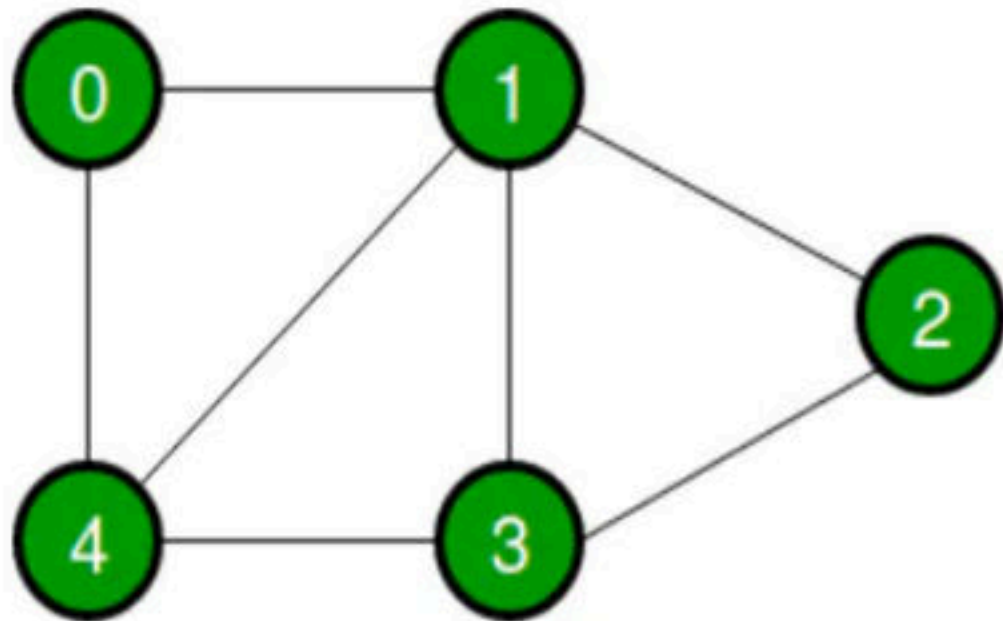
- Graphs are used to represent many real-life applications: Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network.
- Graphs are also used in social networks like LinkedIn, Facebook. For example, in Facebook, each person is represented with a vertex (or node). Each node is a structure and contains information like person id, name, gender and locale.



Representation of Graph in Memory

- Following two are the most commonly used representations of a graph.
 - Adjacency Matrix
 - Adjacency List
- There are other representations also like, Incidence Matrix and Incidence List.
The choice of the graph representation is situation specific. It totally depends on the type of operations to be performed and ease of use.

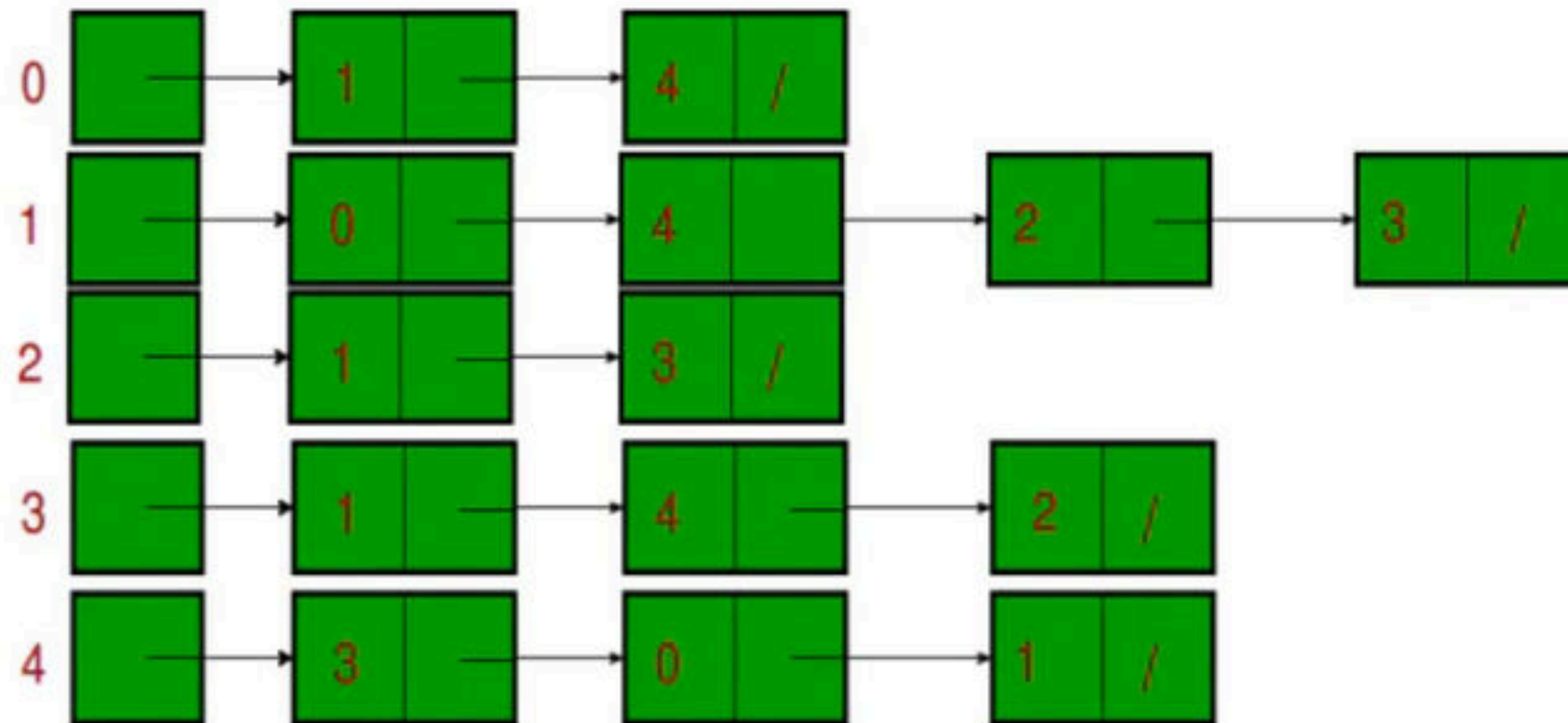
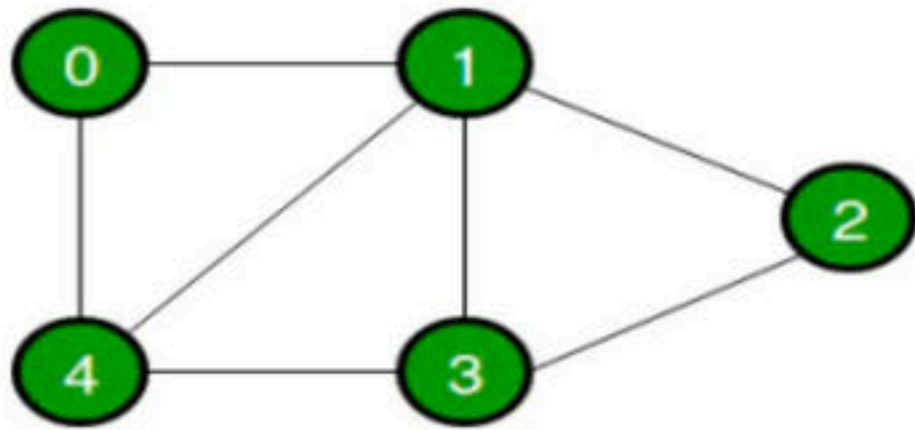
- **Adjacency Matrix:** Adjacency Matrix is a 2D array of size $V \times V$ where V is the number of vertices in a graph. Let the 2D array be $adj[][]$, a slot $adj[i][j] = 1$ indicates that there is an edge from vertex i to vertex j .
- Adjacency matrix for undirected graph is always symmetric.
- Adjacency Matrix is also used to represent weighted graphs. If $adj[i][j] = w$, then there is an edge from vertex i to vertex j with weight w .



	0	1	2	3	4
0					
1					
2					
3					
4					

- **Pros:** Representation is easier to implement and follow. Removing an edge takes $O(1)$ time. Queries like whether there is an edge from vertex 'u' to vertex 'v' are efficient and can be done $O(1)$.
- **Cons:** Consumes more space $O(V^2)$. Even if the graph is sparse(contains less number of edges), it consumes the same space. Adding a vertex is $O(V^2)$ time.

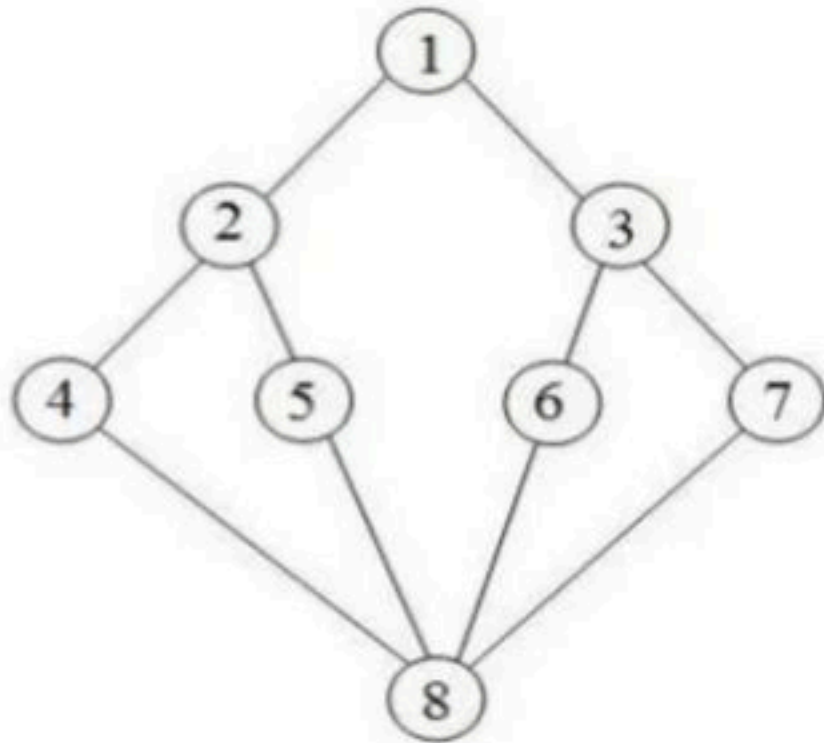
- **Adjacency List:** An array of lists is used. Size of the array is equal to the number of vertices. Let the array be `array[]`. An entry `array[i]` represents the list of vertices adjacent to the i th vertex. This representation can also be used to represent a weighted graph. The weights of edges can be represented as lists of pairs.



Break

Graph Traversal

- Traversal means visiting all the nodes of a graph.
- Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree.
- The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a Boolean visited array.



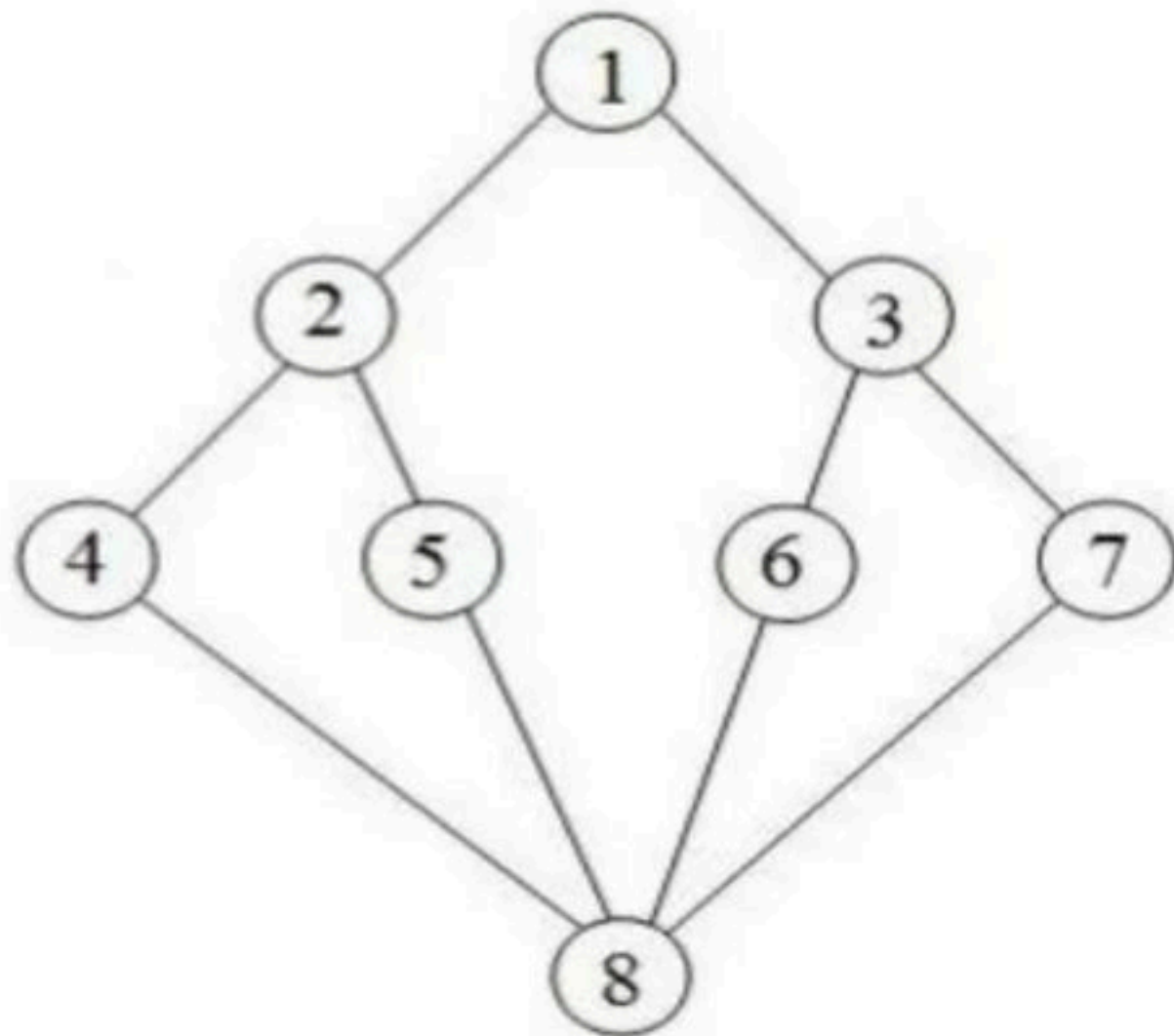
Q Which of the following are valid and invalid DFS traversal sequence

a) 1, 3, 7, 8, 5, 2, 4, 6

b) 1, 2, 5, 8, 6, 3, 7, 4

c) 1, 3, 6, 7, 8, 5, 2, 4

d) 1, 2, 4, 5, 8, 6, 7, 3



- A standard DFS implementation puts each vertex of the graph into one of two categories:
 - Visited
 - Not Visited
- The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

- The DFS algorithm works as follows:
 - Start by putting any one of the graph's vertices on top of a stack.
 - Take the top item of the stack and add it to the visited list.
 - Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of stack.
 - Keep repeating steps 2 and 3 until the stack is empty.

DFS(v)

{

visited(v) = 1

For all x adjacent to v

{

if (x is not visited)

DFS(x)

}

}


```
DFS-iterative (G, s)
{
    let S be stack
    Push( s )
    while ( S is not empty)
    {
        v = pop(S)
        if v is not marked as visited
        {
            mark v as visited
            for all neighbors w of v in Graph G:
            {
                if w is not marked as visited:
                    push( w )
            }
        }
    }
}
```

DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1   $time = time + 1$                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$           // explore edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$              // blacken  $u$ ; it is finished
9   $time = time + 1$ 
10  $u.f = time$ 
```

What is the running time of DFS? The loops on lines 1–3 and lines 5–7 of DFS take time $\Theta(V)$, exclusive of the time to execute the calls to DFS-VISIT. As we did for breadth-first search, we use aggregate analysis. The procedure DFS-VISIT is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-VISIT is invoked must be white and the first thing DFS-VISIT does is paint vertex u gray. During an execution of DFS-VISIT(G, v), the loop on lines 4–7 executes $|Adj[v]|$ times. Since

$$\sum_{v \in V} |Adj[v]| = \Theta(E),$$

the total cost of executing lines 4–7 of DFS-VISIT is $\Theta(E)$. The running time of DFS is therefore $\Theta(V + E)$.

Break

Q Consider the following sequence of nodes for the undirected graph given below.

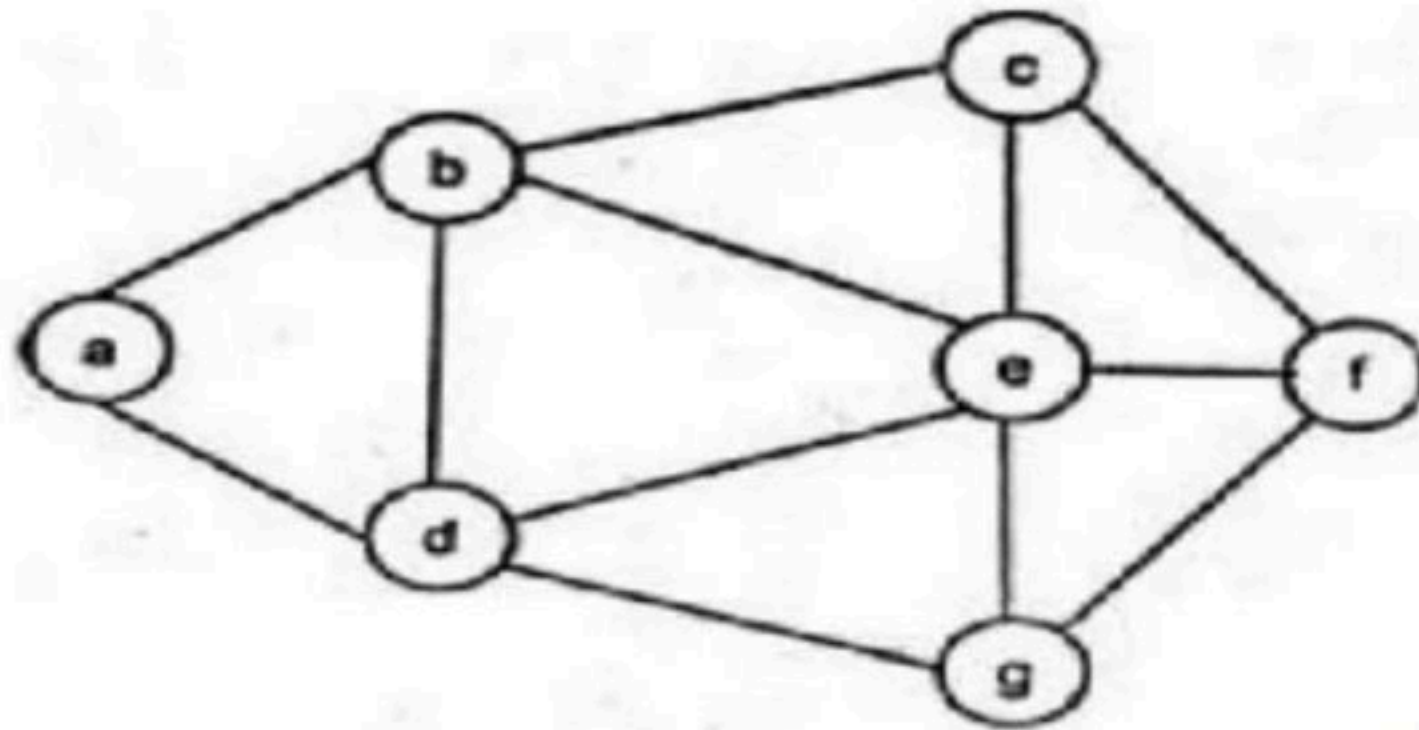
1) a b e f d g c

2) a b e f c g d

3) a d g e b c f

4) a d b c g e f

A Depth First Search (DFS) is started at node a. The nodes are listed in the order they are first visited. Which all of the above is (are) possible output(s)? **(Gate-2008) (2 Marks)**



- (A)** 1 and 3 only
- (B)** 2 and 3 only
- (C)** 2, 3 and 4 only
- (D)** 1, 2, and 3

Q Consider the following graph
Among the following sequences

I) a b e g h f

II) a b f e h g

III) a b f h g e

IV) a f g h b e

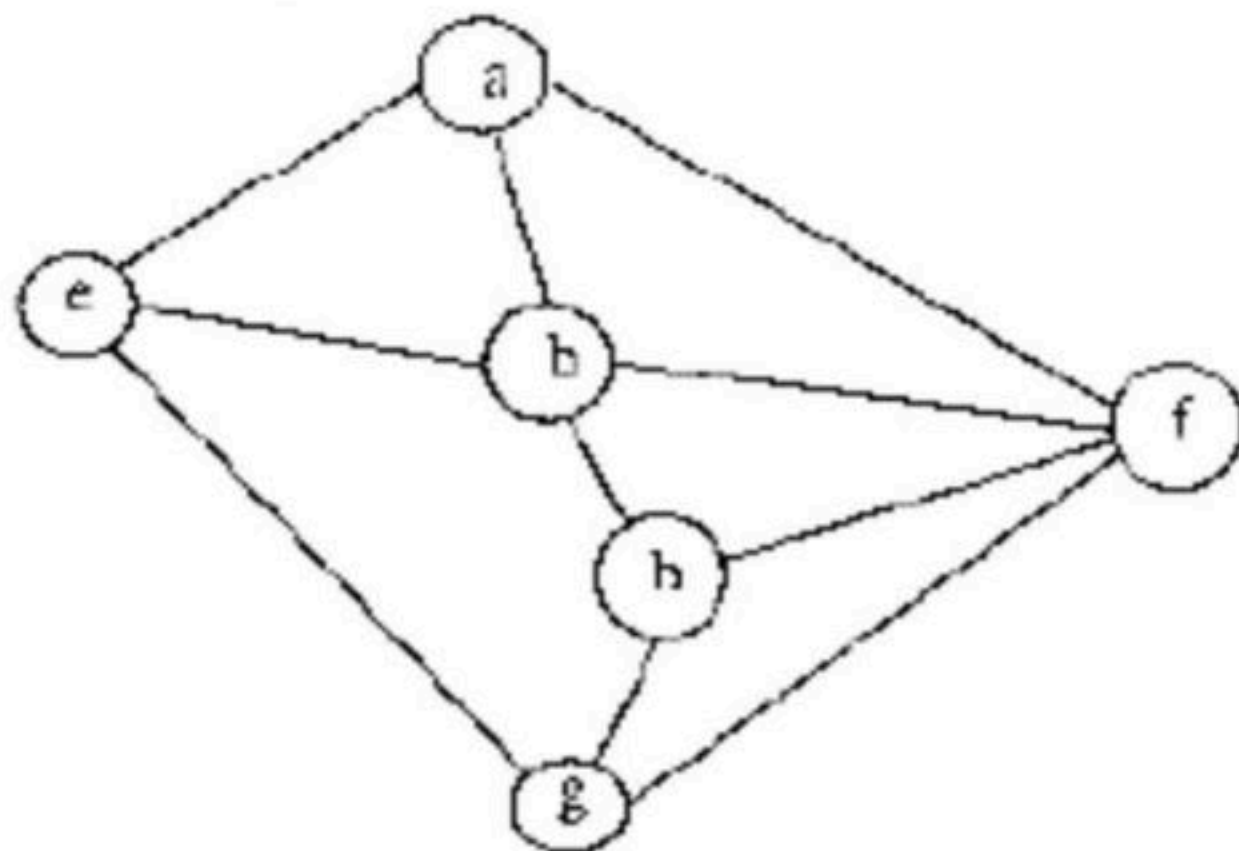
Which are depth first traversals of the above graph? **(GATE-2003) (1 Marks)**

(A) I, II and IV only

(B) I and IV only

(C) II, III and IV only

(D) I, III and IV only



Q Consider the following graph
Among the following sequences

I) a b e g h f

II) a b f e h g

III) a b f h g e

IV) a f g h b e

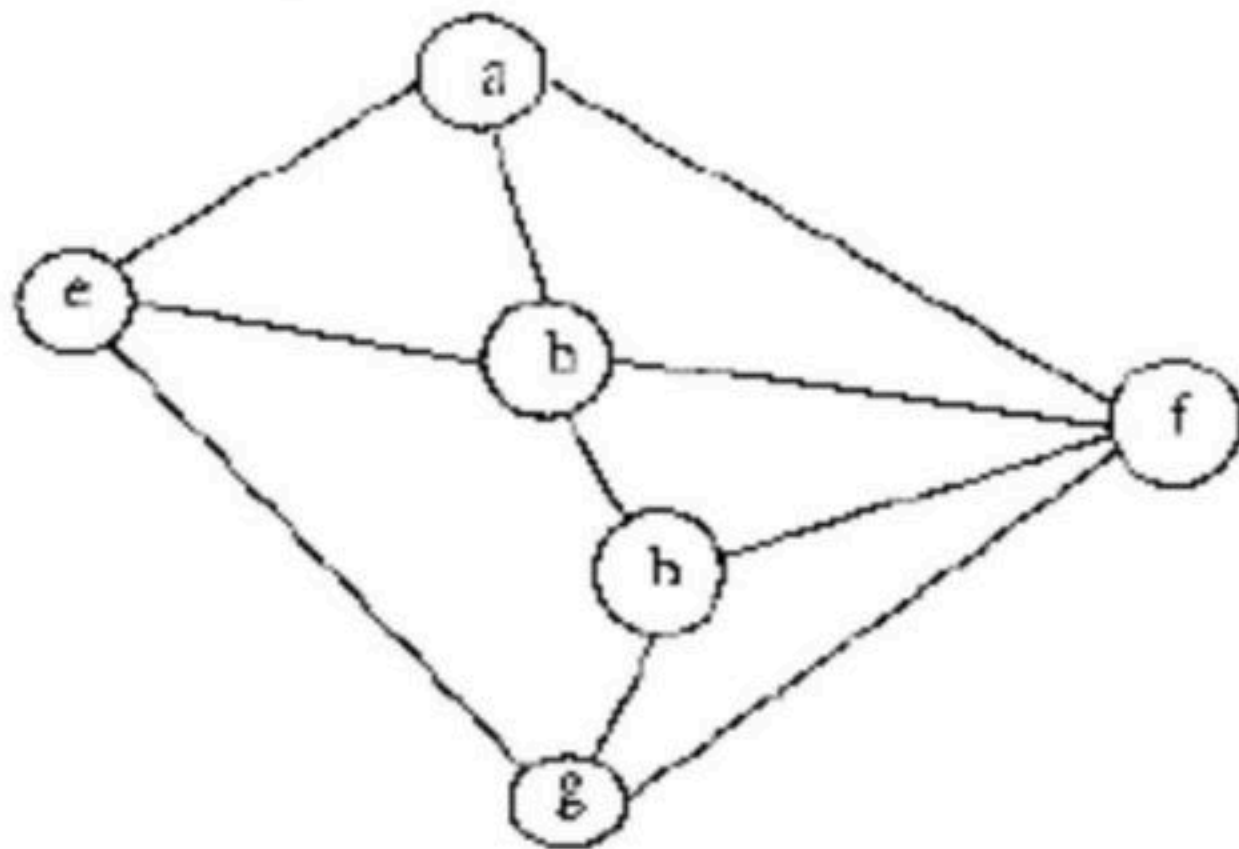
Which are depth first traversals of the above graph? **(GATE-2003) (1 Marks)**

(A) I, II and IV only

(B) I and IV only

(C) II, III and IV only

(D) I, III and IV only



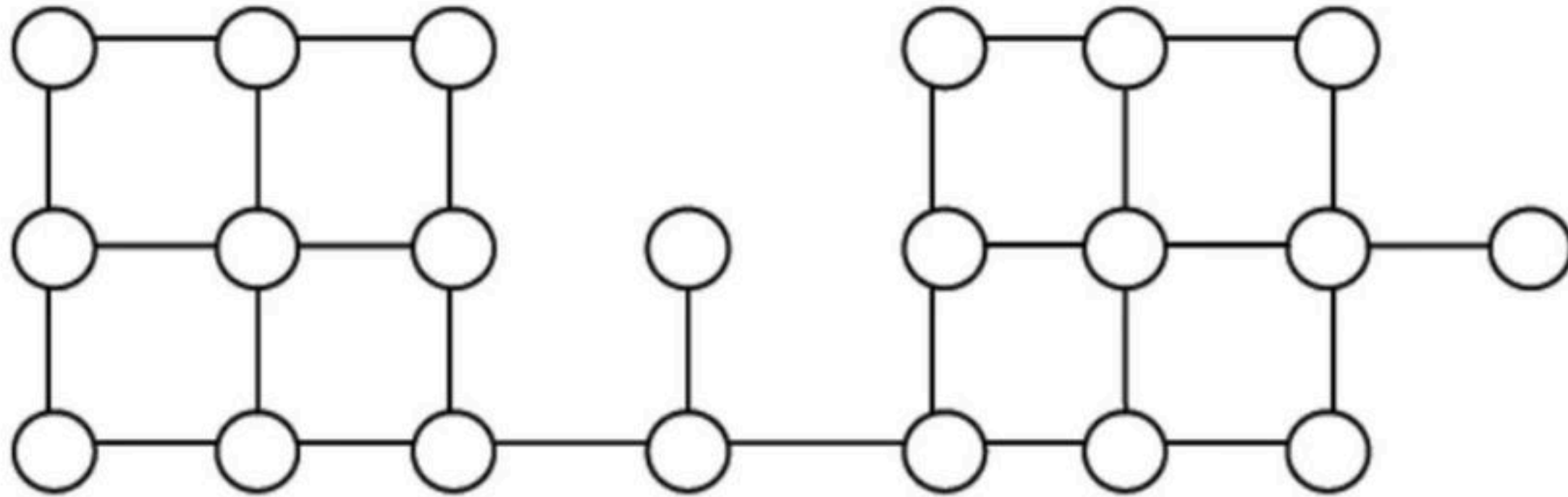
Break

Q.29 An *articulation point* in a connected graph is a vertex such that removing the vertex and its incident edges disconnects the graph into two or more connected components. Let T be a DFS tree obtained by doing DFS in a connected undirected graph G .

Which of the following options is/are correct?

- (a) Root of T is an articulation point in G if and only if it has 2 or more children.
- (b) A leaf of T can be an articulation point in G .
- (c) Root of T can never be an articulation point in G .
- (d) If u is an articulation point in G such that x is an ancestor of u in T and y is a descendent of u in T , then all paths from x to y in G must pass through u .

Q Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is _____. **(Gate-2014) (2 Marks)**



Q Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time on Depth First Search of G ? Assume that the graph is represented using adjacency matrix. **(Gate-2014) (1 Marks)**

(A) $O(n)$

(B) $O(m+n)$

(C) $O(n^2)$

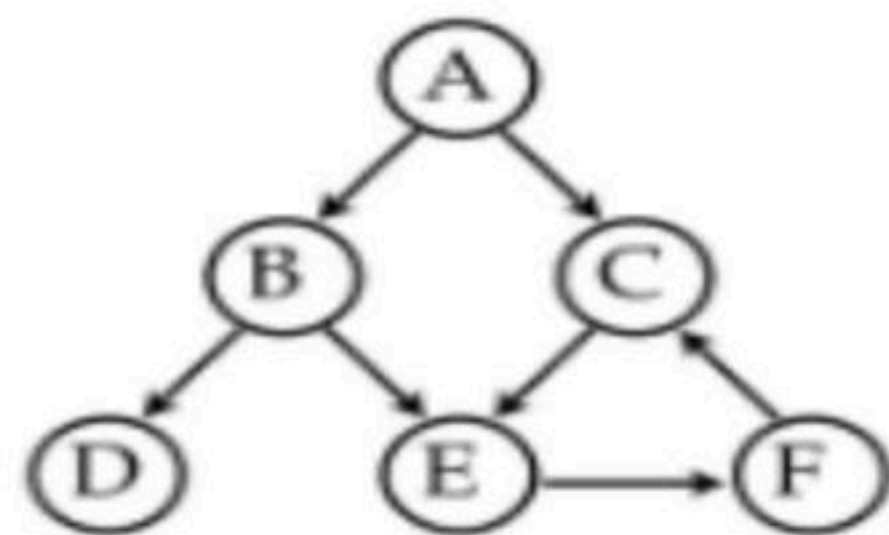
(D) $O(mn)$

Q Let T be a depth first search tree in an undirected graph G . Vertices u and n are leaves of this tree T . The degrees of both u and n in G are at least 2. which one of the following statements is true? **(Gate-2006) (2 Marks)**

- (A)** There must exist a vertex w adjacent to both u and n in G
- (B)** There must exist a vertex w whose removal disconnects u and n in G
- (C)** There must exist a cycle in G containing u and n
- (D)** There must exist a cycle in G containing u and all its neighbors in G .

Depth ion travels of the following directed graph is :

(UGC - June - 2007)



(A) A B C D E F

(C) A C E B D F

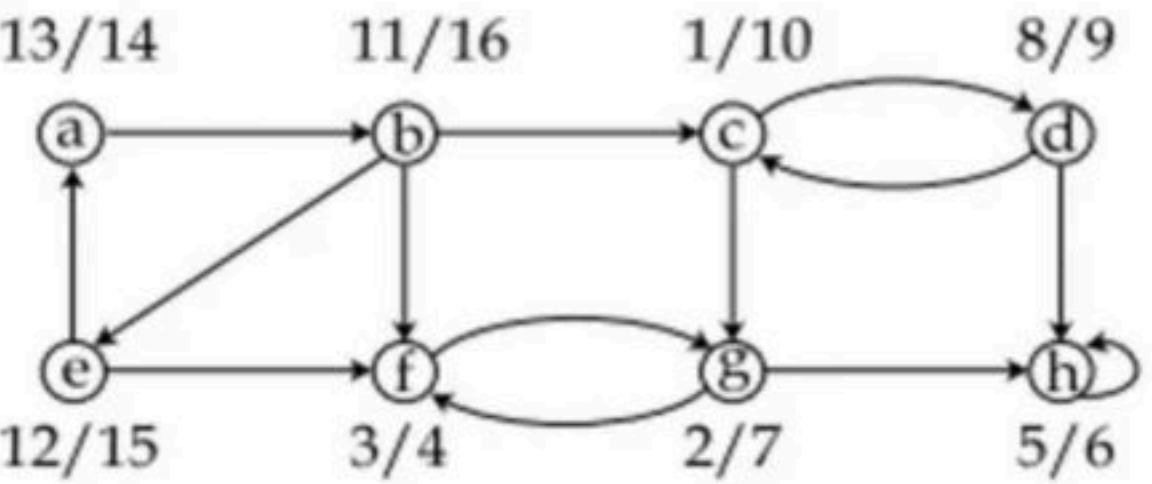
(B) A B D E F C

(D) None of the above

Q Let G be an undirected graph. Consider a depth-first traversal of G , and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statements is always true? (GATE-2000) (2 Marks)

- (A) $\{u, v\}$ must be an edge in G , and u is a descendant of v in T
- (B) $\{u, v\}$ must be an edge in G , and v is a descendant of u in T
- (C) If $\{u, v\}$ is not an edge in G then u is a leaf in T
- (D) If $\{u, v\}$ is not an edge in G then u and v must have the same parent in T

In the following graph, discovery time stamps and finishing time stamps of Depth First Search (DFS) are shown as x/y , where x is discovery time stamp and y is finishing time stamp.



It shows which of the following depth first forest ?

- (1) {a, b, e} {c, d, f, g, h}

(2) {a, b, e} {c, d, h} {f, g}
- (3) {a, b, e} {f, g} {c, d} {h}

(4) {a, b, c, d} {e, f, g} {h}

Break

Breadth First Traversal (or Search)

- Breadth First Traversal (or Search) for a graph is similar to Breadth First Traversal of a tree. The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again.
- To avoid processing a node more than once, we use a Boolean visited array. For simplicity, it is assumed that all vertices are reachable from the starting vertex, i.e. the graph is connected

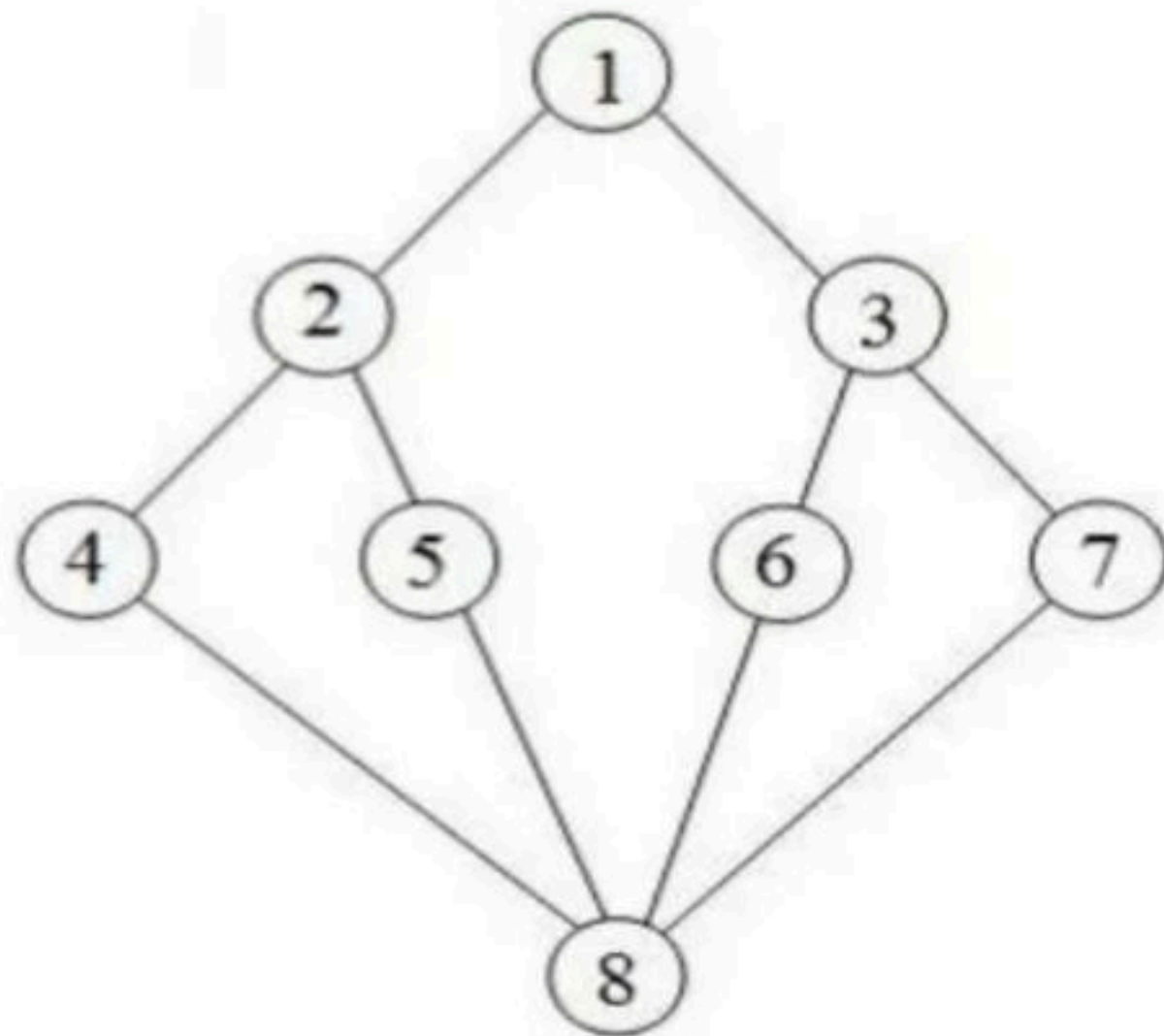
Q Which of the following are valid and invalid BFS traversal sequence

a) 1, 3, 2, 5, 4, 7, 6, 8

b) 1, 3, 2, 7, 6, 4, 5, 8

c) 1, 2, 3, 5, 4, 7, 6, 8

d) 1, 2, 3, 7, 5, 6, 4, 8



```
BFS(v)
{
    visited(v) = 1
    insert[V,Q]
    While(Q != Phi)
    {
        u = Delete(Q);
        for all x adjacent to u
        {
            if (x is not visited)
            {
                visited(x) = 1
                insert(x,Q)
            }
        }
    }
}
```


BFS(G, s)

```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

Before proving the various properties of breadth-first search, we take on the somewhat easier job of analyzing its running time on an input graph $G = (V, E)$. We use aggregate analysis, as we saw in Section 17.1. After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once. The operations of enqueueing and dequeuing take $O(1)$ time, and so the total time devoted to queue operations is $O(V)$. Because the procedure scans the adjacency list of each vertex only when the vertex is dequeued, it scans each adjacency list at most once. Since the sum of the lengths of all the adjacency lists is $\Theta(E)$, the total time spent in scanning adjacency lists is $O(E)$. The overhead for initialization is $O(V)$, and thus the total running time of the BFS procedure is $O(V + E)$. Thus, breadth-first search runs in time linear in the size of the adjacency-list representation of G .



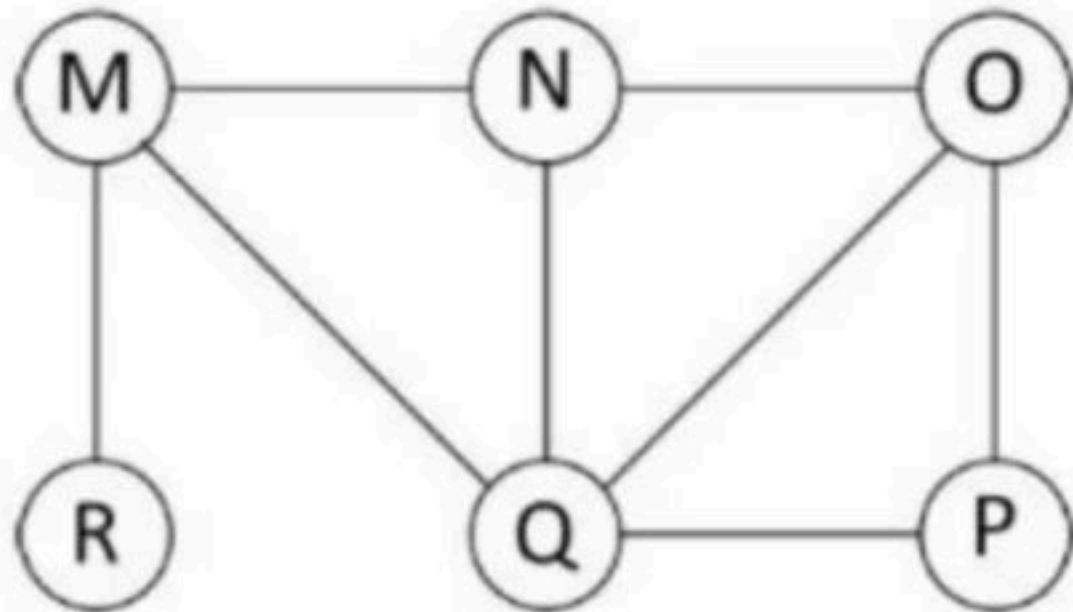
Q Breath First Search (BFS) has been implemented using queue data structure. Which one of the following is a possible order of visiting the nodes in the graph above? **(Gate-2017) (1 Marks)**

a) MNOPQR

b) NQMPOR

c) QMNROP

d) POQNMNR



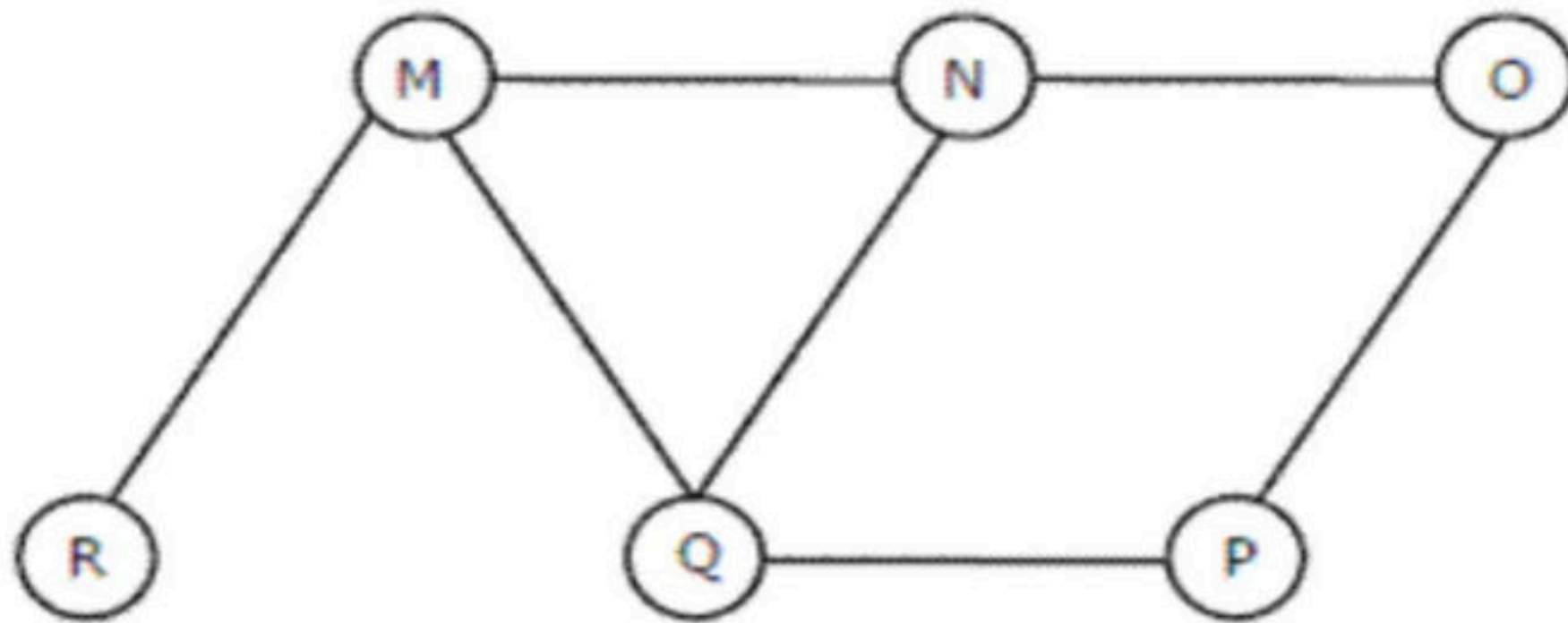
Q The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is **(Gate-2008)** (1 Marks)

(A) MNOPQR

(B) NQMPOR

(C) QMNPRO

(D) QMNPOR



Break

Q.15 Consider a complete binary tree with 7 nodes. Let A denote the set of first 3 elements obtained by performing Breadth-First Search (BFS) starting from the root. Let B denote the set of first 3 elements obtained by performing Depth-First Search (DFS) starting from the root. The value of $|A - B|$ is _____. **(GATE-2021)**

Q Breadth First Search (BFS) is started on a binary tree beginning from the root vertex. There is a vertex t at a distance four from the root. If t is the n -th vertex in this BFS traversal, then the maximum possible value of n is _____ **(Gate-2016) (2 Marks)**

Q Let $G = (V, E)$ be a simple undirected graph, and s be a particular vertex in it called the source. For $x \in V$, let $d(x)$ denote the shortest distance in G from s to x . A breadth first search (BFS) is performed starting at s . Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T , then which one of the following CANNOT be the value of $d(u) - d(v)$? **(Gate-2015)(2 Marks)**

(A) -1

(B) 0

(C) 1

(D) 2

Q Consider the tree arcs of a BFS traversal from a source node W in an unweighted, connected, undirected graph. The tree T formed by the tree arcs is a data structure for computing. **(Gate-2014) (2 Marks)**

- a)** the shortest path between every pair of vertices.
- b)** the shortest path from W to every vertex in the graph.
- c)** the shortest paths from W to only those nodes that are leaves of T .
- d)** the longest path in the graph

Q Level order traversal of a rooted tree can be done by starting from the root and performing **(Gate-2004) (1 Marks)**

(A) preorder traversal

(B) inorder traversal

(C) depth first search

(D) breadth first search

Q Consider an undirected unweighted graph G . Let a breadth-first traversal of G be done starting from a node r . Let $d(r, u)$ and $d(r, v)$ be the lengths of the shortest paths from r to u and v respectively, in G . If u is visited before v during the breadth-first traversal, which of the following statements is correct? **(GATE-2001) (2 Marks)**

- (A)** $d(r, u) < d(r, v)$ **(B)** $d(r, u) > d(r, v)$ **(C)** $d(r, u) \leq d(r, v)$ **(D)** None of the above

Break

Q Let G be a simple undirected graph. Let T_D be a depth first search tree of G . Let T_B be a breadth first search tree of G . Consider the following statements.

(I) No edge of G is a cross edge with respect to T_D . (A cross edge in G is between two nodes neither of which is an ancestor of the other in T_D).

(II) For every edge (u, v) of G , if u is at depth i and v is at depth j in T_B , then $|i - j| = 1$.

Which of the statements above must necessarily be true? **(Gate-2018) (2 Marks)**

- a) I only b) II only c) Both I and II d) Neither I nor II

Q The most efficient algorithm for finding the number of connected components in an undirected graph on n vertices and m edges has time complexity. **(Gate-2008)(1 Marks)**

a) $O(n)$

b) $O(m)$

c) $O(m+n)$

d) $O(m.n)$

Break

Special Class(You Tube)				
Date	Day	Topic	Timing	Duration
01-11-2021	Monday			
02-11-2021	Tuesday			
03-11-2021	Wednesday			
04-11-2021	Thursday			
05-11-2021	Friday			
06-11-2021	Saturday			
07-11-2021	Sunday			
08-11-2021	Monday			
09-11-2021	Tuesday			
10-11-2021	Wednesday			
11-11-2021	Thursday			
12-11-2021	Friday	Problem Solving In DBMS Part-1	7:00 pm to 8:00 pm	1
13-11-2021	Saturday	Problem Solving In DBMS Part-2	7:00 pm to 8:00 pm	1
14-11-2021	Sunday	Problem Solving In DBMS Part-3	7:00 pm to 8:00 pm	1
15-11-2021	Monday			
16-11-2021	Tuesday			
17-11-2021	Wednesday			
18-11-2021	Thursday			
19-11-2021	Friday	Problem Solving In Data Structure & Algo Part-1	7:00 pm to 8:00 pm	1
20-11-2021	Saturday	Problem Solving In Data Structure & Algo Part-2	7:00 pm to 8:00 pm	1
21-11-2021	Sunday	Problem Solving In Data Structure & Algo Part-3	7:00 pm to 8:00 pm	1
22-11-2021	Monday			
23-11-2021	Tuesday			
24-11-2021	Wednesday			
25-11-2021	Thursday			
26-11-2021	Friday	Problem Solving In Computer Network Part-1	7:00 pm to 8:00 pm	1
27-11-2021	Saturday	Problem Solving In Computer Network Part-2	7:00 pm to 8:00 pm	1
28-11-2021	Sunday	Problem Solving In Computer Architecture Part-1	7:00 pm to 8:00 pm	1
29-11-2021	Monday	Problem Solving In Computer Architecture Part-2	7:00 pm to 8:00 pm	1
30-11-2021	Tuesday			